

BUILD A BITTORRENT CLIENT with Modern Node.js

A beginner-friendly, updated version of Allen Kim's 2016 tutorial

.torrent file	UDP tracker	TCP peers	pieces + blocks	finished file
----------------------	--------------------	------------------	------------------------	----------------------

Edition verified: 2 July 2026

Target runtime: Node.js 24 LTS | bencode 4.0.1 | ECMAScript modules

Educational use only - download and share only content you are legally allowed to use.

About this guide

This guide keeps the learning path and subject coverage of the original tutorial, but rewrites the code for a current Node.js environment and explains the missing background topics in one place. It is deliberately slower and more detailed than a normal programming article.

What you will build

A small command-line BitTorrent v1 downloader that reads a .torrent file, contacts a UDP tracker, connects to peers over TCP, requests blocks, and writes a single output file. It is a learning client, not a production replacement for qBittorrent, Transmission, or similar software.

What has been modernised

2016 tutorial	Modern guide	Why it changed
CommonJS: require() and module.exports	ES modules: import and export	The bencode 4.x package is an ES module, and modern Node projects commonly use the standard module syntax.
Native bigint package	Built-in BigInt and Buffer.writeBigUInt64BE()	Node can now represent and write the 64-bit unsigned fields used by the UDP tracker protocol without a native add-on.
url.parse()	new URL()	The WHATWG URL API is the current, clear way to parse tracker URLs.
Callback-only control flow	Promises and async/await around tracker requests	The protocol remains event-driven, but the top-level flow is easier to read and errors are easier to propagate.
Weak response checking	Action, length, and transaction-ID validation	UDP can receive unrelated or malformed packets; responses must be matched to requests.
Incorrect announce offsets in the article code	Offsets match BEP 15	The event, IP, key, num_want, port, interval, leechers, and seeders fields now start at the specified byte positions.
Assumed one TCP data event equals one message	A persistent receive buffer extracts complete messages	TCP is a byte stream. A data event may contain a fragment, one message, or several messages.
No test suite	Built-in node:test tests	Protocol byte layouts can be checked without another testing package.

Scope and deliberate limitations

- BitTorrent v1 metainfo only. BitTorrent v2 and hybrid torrents use additional rules and SHA-256 based structures.
- Single-file torrents only. Multi-file layout is explained as an extension.
- UDP trackers only. HTTP/HTTPS trackers, DHT, PEX, LSD, magnet links, and WebRTC are outside the core build.
- IPv4 compact peer lists only. IPv6 compact peers use a different representation.
- Download-only. This client does not upload pieces or participate as a full seeder.
- One outstanding block per peer. This is intentionally simple, but slower than request pipelining.
- The final learning build does not yet verify each completed piece hash before accepting it. The integrity upgrade is explained in the improvements chapter and should be implemented before treating the program as robust.

Legal and network safety

Use a small, legal test torrent. Public peer IP addresses are untrusted input. Run this code as a normal user, not as an administrator, and do not open an inbound router port for this tutorial.

How to use the guide

1. Read Chapters 1-4 before coding. They explain the protocol, Buffer objects, byte order, and bencoding.
2. Create the project in Chapter 5 and add each file in order.
3. Run npm test after the protocol-building chapters. Byte-level bugs are easier to fix before trying a live tracker.
4. Use the full source listings in Appendix A to compare your files.
5. Use the troubleshooting chapter when a tracker or peer fails. A failure does not always mean your code is wrong; public trackers and peers come and go.

Contents

Part I - Foundations

- 1. What BitTorrent is and what this client will do
- 2. Modern Node.js setup and project structure
- 3. Bits, bytes, hexadecimal, encodings, and Buffer
- 4. Bencoding and the anatomy of a .torrent file

Part II - Discovering peers through a tracker

- 5. Open and inspect a torrent
- 6. UDP, dgram, tracker URLs, and the two-request flow
- 7. Build and parse the UDP connect exchange
- 8. Build and parse the UDP announce exchange

Part III - Talking to peers and downloading data

- 9. TCP, the peer-wire protocol, and the handshake
- 10. Peer message types and message builders
- 11. Reassemble complete messages from a TCP stream
- 12. Pieces, blocks, queues, and failed requests
- 13. Connect to peers, request blocks, and write the file
- 14. Run the client safely

Part IV - Confidence, diagnosis, and next steps

- 15. Test the protocol code with node:test
- 16. Common errors and troubleshooting
- 17. Improvements and a production-ready roadmap

Appendices

- A. Complete working source code
- B. Protocol byte-layout cheat sheets
- C. Glossary
- D. Further reading and primary sources

1. What BitTorrent is and what this client will do

1.1 The simple mental model

A normal web download usually asks one server for one file. BitTorrent divides a file into numbered pieces and lets many computers exchange those pieces. A computer participating in the same torrent is called a peer.

The two halves of this project

First, ask a tracker for peer addresses. Second, connect directly to those peers and request file blocks. The tracker does not normally send the file itself.

Step	Action	Meaning
1	Read the .torrent file	Learn the tracker address, file name, size, piece size, and the expected SHA-1 hash for every piece.
2	Contact a tracker	Send the torrent info hash and receive IP address + port pairs for peers.
3	Open TCP connections	Connect directly to a selection of peers.
4	Handshake	Prove that both sides are talking about the same torrent.
5	Learn what each peer has	Process have and bitfield messages.
6	Request blocks	Ask an unchoking peer for a block within a piece.
7	Write blocks at exact offsets	Place each received block in the correct position in the output file.
8	Finish	Stop when every expected block has been received.

1.2 Tracker, peer, seeder, leecher, piece, and block

Term	Beginner meaning
Tracker	A coordination service. It tells clients where peers may be found.
Peer	Another BitTorrent client connected to the same torrent swarm.
Seeder	A peer that has all of the torrent data.
Leecher	A peer that is still downloading. It may also upload pieces it already has.
Swarm	All peers participating in one torrent.
Piece	A fixed-size region of the torrent data, except the final piece may be shorter.
Block	A smaller request-sized region inside a piece. This guide uses 16 KiB blocks.
Info hash	A 20-byte SHA-1 identifier made from the bencoded info dictionary in a v1 torrent.
Peer ID	A 20-byte identifier for one running client instance.

1.3 Why the project is educationally useful

- You work with real binary protocols rather than JSON-only APIs.
- You learn TCP and UDP differences through code.
- You practise asynchronous events, buffers, offsets, hashing, files, queues, and tests.
- You see why network code must validate lengths and identifiers instead of trusting input.

- You build a larger project from small modules instead of putting everything in index.js.

1.4 What happens on the wire

Most values in BitTorrent protocol messages are not sent as readable text. They are bytes with agreed positions and sizes. For example, a four-byte unsigned integer may occupy bytes 8 through 11 of a message. Both sides must agree that those four bytes mean the same thing.

A five-byte peer message shown in hexadecimal

```
00 00 00 01 02
^ ^ ^ ^ ^ ^ ^ ^ ^ ^
length = 1 id = 2 (interested)
```

The first four bytes are the big-endian integer 1. That says one byte follows the length prefix. The next byte has value 2, which means interested.

2. Modern Node.js setup and project structure

2.1 Recommended versions

This edition targets Node.js 24 LTS. Node.js 26 is the Current release line at the time of verification, but an LTS release is the safer choice for a learning project because it receives long-term maintenance. The code uses only standard Node APIs plus bencode 4.0.1.

Version rule

Use Node.js 24.x LTS for the steps in this guide. A newer Node release will probably work, but the tested target and package engines field are Node 24 or later.

2.2 Check your tools

Run in Terminal, PowerShell, or Command Prompt

```
node --version
npm --version
```

The first command should print a version beginning with v24 when you use the recommended LTS line. npm is included with Node.js. You also need a text editor such as Visual Studio Code.

2.3 Create the project

Create the folders

```
mkdir modern-torrent-client
cd modern-torrent-client
mkdir src test
```

The src folder holds application modules. The test folder holds automated checks. The project root holds package.json and index.js.

2.4 package.json

package.json

```
1 | {
2 |   "name": "modern-bittorrent-client-tutorial",
3 |   "version": "1.0.0",
4 |   "private": true,
5 |   "description": "A small BitTorrent v1 client built for learning purposes.",
6 |   "type": "module",
7 |   "scripts": {
8 |     "start": "node index.js",
9 |     "test": "node --test"
10 |  },
11 |   "engines": {
12 |     "node": ">=24"
13 |  },
14 |   "dependencies": {
15 |     "bencode": "4.0.1"
16 |  }
17 | }
```

Line-by-line explanation

Property	Meaning
name	The package name used by npm. It does not have to match the folder exactly.
version	Your project version. 1.0.0 is a normal starting value.

Property	Meaning
private	Prevents accidental publishing to the public npm registry.
description	A human-readable summary.
type: module	Makes .js files use ECMAScript module syntax, so import and export work as written.
scripts.start	Lets you run <code>npm start -- <arguments></code> , although <code>direct node index.js</code> is clearer while learning.
scripts.test	Runs Node's built-in test runner.
engines.node	Documents that this guide targets Node 24 or later.
dependencies.bencode	Pins the bencode package to 4.0.1 for repeatable installation.

2.5 Install the dependency

Install exactly what package.json specifies

```
npm install
```

npm creates `node_modules` and `package-lock.json`. The lock file records the exact dependency tree installed on your computer. Keep `package-lock.json` in source control if you put the project on GitHub.

2.6 Final folder layout

Project tree

```
modern-torrent-client/
|-- package.json
|-- package-lock.json
|-- index.js
|-- src/
|   |-- torrent-parser.js
|   |-- tracker.js
|   |-- util.js
|   |-- message.js
|   |-- Queue.js
|   |-- Pieces.js
|   |-- download.js
|-- test/
|   |-- protocol.test.js
```

2.7 ES modules in plain English

Modern import/export example

```
import fs from 'node:fs';
import bencode from 'bencode';

export function openTorrent(filePath) {
  // ...
}
```

`import` brings a value from another module into the current file. The `node:` prefix makes it obvious that `fs` is a built-in Node module. `export` makes a function available to other files. A relative import must include the `.js` extension in this project, for example `./src/download.js`.

3. Bits, bytes, hexadecimal, encodings, and Buffer

3.1 Why this chapter matters

A BitTorrent client spends much of its time converting between meaningful values and exact byte layouts. Buffer is the Node.js type used for those bytes. Understanding it removes most of the mystery from tracker and peer messages.

3.2 Bits and bytes

A bit is one binary digit: 0 or 1. Eight bits make one byte. A byte can represent 256 different patterns, from 00000000 to 11111111, which are the decimal values 0 through 255.

Binary	Decimal	Hexadecimal	Comment
00000000	0	00	All bits off.
00000001	1	01	The lowest bit is on.
00001010	10	0A	Hex uses A for decimal 10.
01111111	127	7F	Largest positive signed 8-bit value.
11111111	255 unsigned / -1 signed	FF	The meaning depends on signedness.

3.3 Why hexadecimal is used

Binary is accurate but long. Hexadecimal groups each four bits into one character, so one byte is always shown by two hex characters. The byte 11110000 becomes F0. Network debugging tools commonly show packets in hex because offsets and byte boundaries remain easy to see.

3.4 Text encodings are rules, not decorations

A string such as hello is abstract text. To send it through a socket, it must be encoded as bytes. UTF-8 is the normal text encoding in Node.js. ASCII is a smaller historical character set whose common English characters have the same byte values as UTF-8.

Text and bytes

```
import { Buffer } from 'node:buffer';

const text = Buffer.from('hello', 'utf8');
console.log(text);           // <Buffer 68 65 6c 6c 6f>
console.log(text.toString('hex')); // 68656c6c6f
console.log(text.toString('utf8')); // hello
```

The bytes 68 65 6c 6c 6f are hexadecimal values. They are not the characters six, eight, six, five, and so on. The console uses hex merely to display the raw bytes.

3.5 Buffer.alloc, Buffer.from, and Buffer.allocUnsafe

Method	Use it when	Safety
Buffer.alloc(length)	You know how many bytes are required and want them initially set to zero.	Safest default for protocol messages.
Buffer.from(value)	You already have text, an array of byte values, another Buffer, or compatible data.	Copies or encodes existing data.

Method	Use it when	Safety
<code>Buffer.allocUnsafe(length)</code>	You will immediately overwrite every byte and need maximum allocation speed.	May expose old memory content until overwritten. Avoid while learning.

3.6 Buffers are fixed length

Write a 32-bit integer

```
const buffer = Buffer.alloc(4);
buffer.writeUInt32BE(1000, 0);
console.log(buffer); // <Buffer 00 00 03 e8>
```

`Buffer.alloc(4)` creates exactly four bytes. It cannot grow. `writeUInt32BE` writes an unsigned 32-bit integer in big-endian order starting at offset 0. The value 1000 is 0x000003E8, so the four bytes are 00 00 03 E8.

3.7 Signed and unsigned integers

Unsigned values cannot be negative. A 16-bit unsigned integer ranges from 0 to 65,535. Signed formats reserve one bit pattern range for negative values. The UDP announce request uses a signed 32-bit field for `num_want` because -1 means use the tracker default.

The same bits, two interpretations

```
const b = Buffer.alloc(4);
b.writeInt32BE(-1, 0);
console.log(b.toString('hex')); // ffffffff
console.log(b.readInt32BE(0)); // -1
console.log(b.readUInt32BE(0)); // 4294967295
```

3.8 Big-endian and little-endian

Endianness is the order in which a multi-byte number is stored. Network protocols commonly use big-endian, sometimes called network byte order. Big-endian writes the most significant byte first. BitTorrent tracker and peer integers in this guide are big-endian.

Same number, different byte order

```
const b = Buffer.alloc(4);
b.writeUInt32BE(0x12345678, 0);
console.log(b.toString('hex')); // 12345678

b.writeUInt32LE(0x12345678, 0);
console.log(b.toString('hex')); // 78563412
```

3.9 Offsets

An offset is a zero-based byte position. Offset 0 is the first byte. Writing a four-byte integer at offset 8 occupies offsets 8, 9, 10, and 11. The next free offset is 12.

Offset example

```
const message = Buffer.alloc(16);
message.writeUInt32BE(0, 8); // bytes 8..11
message.writeUInt32BE(123, 12); // bytes 12..15
```

Most protocol bugs are offset bugs

Before coding a binary message, write a layout table containing offset, size, type, and meaning. Add the sizes cumulatively. The announce request chapter follows this method.

3.10 copy, subarray, concat, and equals

Operation	Example	Meaning
copy	<code>source.copy(target, 16)</code>	Copies source bytes into target beginning at target offset 16.
subarray	<code>buffer.subarray(8, 16)</code>	Creates a view of bytes 8 through 15. The end offset is excluded.
concat	<code>Buffer.concat([oldData, newData])</code>	Creates one Buffer containing all supplied buffers in order.
equals	<code>a.equals(b)</code>	Returns true when two Buffers contain the same bytes.
toString	<code>buffer.toString('utf8')</code>	Decodes bytes as text. Do this only for fields that are actually text.

3.11 64-bit values and BigInt

JavaScript Number cannot exactly represent every 64-bit integer. Modern JavaScript has BigInt, written with an n suffix. Node Buffer supports reading and writing 64-bit BigInt values.

The UDP tracker protocol ID

```
const b = Buffer.alloc(8);
const protocolId = 0x41727101980n;
b.writeBigUInt64BE(protocolId, 0);
console.log(b.toString('hex')); // 0000041727101980
```

This replaces the old tutorial's manual split into two 32-bit halves and replaces the native bigint package used for torrent sizes.

3.12 Buffer practice

1. Create a 6-byte Buffer.
2. Write IPv4 bytes 127, 0, 0, 1 at offsets 0-3.
3. Write port 6881 as a big-endian unsigned 16-bit integer at offset 4.
4. Read the port back and convert the first four bytes into 127.0.0.1.

One compact IPv4 peer entry

```
const peer = Buffer.alloc(6);
Buffer.from([127, 0, 0, 1]).copy(peer, 0);
peer.writeUInt16BE(6881, 4);

const ip = [...peer.subarray(0, 4)].join('.');
const port = peer.readUInt16BE(4);
console.log({ ip, port });
```

4. Bencoding and the anatomy of a .torrent file

4.1 What bencoding is

Bencoding is the compact data format used by BitTorrent metainfo files and some protocol extensions. It supports four value types: byte strings, integers, lists, and dictionaries. It is not JSON, although decoded data can look similar in a console.

Type	Bencoded form	Decoded meaning
Byte string	4:spam	Four bytes containing spam.
Integer	i42e	The integer 42. i starts it and e ends it.
List	l4:spam4:eggse	A list containing spam and eggs.
Dictionary	d3:cow3:moo4:spam4:eggse	Keys and values: cow -> moo, spam -> eggs.

4.2 Byte strings are length-prefixed

In 4:spam, the 4 describes the number of bytes after the colon, not necessarily the number of visible characters. UTF-8 characters can use more than one byte, so protocol code must think in bytes.

Characters and bytes are not always the same count

```
Buffer.byteLength('hello', 'utf8'); // 5
Buffer.byteLength('£', 'utf8');    // 2
```

4.3 Integers

An integer begins with i and ends with e. The digits between them are base-ten ASCII. Examples are i0e, i123e, and i-7e. Canonical bencoding does not use leading zeros, so i03e is invalid.

4.4 Lists

A list begins with l and ends with e. Its elements are placed one after another. Because every value has a complete representation, a decoder knows exactly where one element finishes and the next begins.

A list containing the byte string spam and integer 42

```
l4:spami42ee
^      ^
list   end
```

4.5 Dictionaries

A dictionary begins with d and ends with e. Keys are bencoded byte strings followed immediately by their values. Canonical dictionaries sort keys by their raw byte-string values. This ordering matters when calculating a v1 info hash.

Dictionary with name and length

```
d4:name9:puppy.jpg6:lengthi124234ee
```

4.6 Decode and encode with bencode 4.0.1

bencode package basics

```
import bencode from 'bencode';

const encoded = Buffer.from('d3:cow3:moo4:spam4:eggse');
const decoded = bencode.decode(encoded);
console.log(decoded.cow.toString('utf8')); // moo

const again = bencode.encode(decoded);
console.log(again.toString('utf8'));
```

The package returns byte strings as Buffer values. That is correct and important: not every byte string is human-readable text. The pieces field, for example, contains raw SHA-1 hashes and must not be decoded as UTF-8.

4.7 A torrent file at a high level

Typical decoded single-file v1 torrent

```
{
  announce: <Buffer ...>,
  'announce-list': [ ... ],
  'creation date': 1462355939,
  'created by': <Buffer ...>,
  info: {
    name: <Buffer ...>,
    length: 124234,
    'piece length': 16384,
    pieces: <Buffer ...>
  }
}
```

Field	Meaning
announce	Primary tracker URL.
announce-list	Optional tiers of alternative tracker URLs.
creation date	Optional Unix timestamp describing when the torrent was created.
created by	Optional name/version of the program that made the torrent.
info.name	Suggested file name for a single-file torrent, or top-level folder for a multi-file torrent.
info.length	File size in bytes for a single-file torrent.
info.files	Array of files for a multi-file torrent. This replaces info.length.
info.piece length	Normal number of bytes in each piece.
info.pieces	All expected v1 piece SHA-1 hashes joined together. Each hash is exactly 20 bytes.

4.8 Why pieces.length / 20 gives the piece count

SHA-1 always returns 20 bytes. If info.pieces contains 400 bytes, it contains $400 / 20 = 20$ piece hashes. The piece at index 0 uses bytes 0-19, index 1 uses bytes 20-39, and so on.

4.9 The info hash

For a v1 torrent, the info hash is SHA-1 applied to the exact bencoded info dictionary. It identifies the torrent when speaking to a tracker or peer. It is not the hash of the downloaded file as one whole object.

Simple educational calculation

```
const encodedInfo = bencode.encode(torrent.info);
const hash = crypto.createHash('sha1').update(encodedInfo).digest();
```

A subtle production detail

The safest production parser hashes the exact raw byte slice occupied by the info dictionary in the original .torrent file. Decode-then-re-encode works for normal canonical torrents, which is what this guide assumes, but a non-canonical file could re-encode differently.

4.10 Single-file versus multi-file torrents

A single-file torrent has `info.length`. A multi-file torrent has `info.files`, where each item contains a length and path components. The bytes still form one continuous logical stream for piece hashing. Supporting multiple output files therefore requires mapping stream offsets across file boundaries.

5. Open and inspect a torrent

5.1 Create src/torrent-parser.js

First version of torrent-parser.js

```

1 | import { Buffer } from 'node:buffer';
2 | import crypto from 'node:crypto';
3 | import fs from 'node:fs';
4 |
5 | import bencode from 'bencode';
6 |
7 | export function openTorrent(filePath) {
8 |   const file = fs.readFileSync(filePath);
9 |   return bencode.decode(file);
10 | }

```

Every line explained

Line or expression	What it does
import { Buffer } from 'node:buffer'	Explicitly imports Node's byte container. Buffer is global in Node, but an explicit import makes the dependency clear.
import crypto from 'node:crypto'	Loads hashing and random-byte functions.
import fs from 'node:fs'	Loads file-system functions.
import bencode from 'bencode'	Loads the installed metainfo encoder/decoder.
export function openTorrent(filePath)	Declares a reusable function and exports it to other modules.
fs.readFileSync(filePath)	Reads the complete .torrent file into a Buffer. Metainfo files are normally small, so a synchronous read keeps the example simple.
bencode.decode(file)	Converts bencoded bytes into JavaScript objects, arrays, numbers, and Buffers.
return	Gives the decoded torrent object back to the caller.

5.2 Inspect without corrupting binary fields

Temporary inspection script

```

import { openTorrent } from './src/torrent-parser.js';

const torrent = openTorrent(process.argv[2]);
console.dir(torrent, { depth: null });

```

Run it

```
node index.js path/to/legal-test.torrent
```

Do not call `JSON.stringify` and assume the result represents all binary data meaningfully. Buffers such as `info.pieces` are raw hashes. Convert only known text fields with `toString(utf8)`.

5.3 Text helper functions

Safe text conversion and scope check

```

export function torrentName(torrent) {
  return Buffer.from(torrent.info.name).toString('utf8');
}

export function isSingleFileTorrent(torrent) {
  return torrent.info.files === undefined;
}

```

```
}
```

`Buffer.from` makes sure the value is treated as a `Buffer` before decoding it. `isSingleFileTorrent` returns `true` when the files array is absent. The final program stops with a clear message for multi-file torrents instead of writing the logical stream into one incorrect file.

5.4 Read all tracker URLs

Gather primary and alternative trackers

```
export function trackerUrls(torrent) {
  const urls = [];

  if (torrent.announce) {
    urls.push(Buffer.from(torrent.announce).toString('utf8'));
  }

  if (Array.isArray(torrent['announce-list'])) {
    for (const tier of torrent['announce-list']) {
      const entries = Array.isArray(tier) ? tier : [tier];
      for (const entry of entries) {
        urls.push(Buffer.from(entry).toString('utf8'));
      }
    }
  }

  return [...new Set(urls)];
}
```

`announce-list` is normally a list of tiers, and each tier is a list of URLs. The code tolerates a less regular shape by wrapping non-array tiers. `Set` removes duplicates.

5.5 Choose a UDP tracker

Select the protocol supported by this guide

```
export function chooseUdpTracker(torrent) {
  const url = trackerUrls(torrent).find((candidate) =>
    candidate.startsWith('udp://'),
  );

  if (!url) {
    throw new Error(
      'No UDP tracker was found in this torrent. ' +
      'This learning client does not yet support HTTP trackers or DHT.',
    );
  }

  return url;
}
```

6. UDP, dgram, tracker URLs, and the two-request flow

6.1 UDP versus TCP

Property	UDP	TCP
Connection	No persistent connection handshake at the transport layer.	Creates a connection before application messages are exchanged.
Delivery	Packets may be lost, duplicated, or arrive out of order.	Provides an ordered byte stream or reports a connection failure.
Message boundaries	Each datagram arrives as a datagram.	No application-message boundaries; data may be split or combined.
Used here for	Tracker connect and announce exchanges.	Direct peer-wire connections and file blocks.
Node module	node:dgram	node:net

6.2 A minimal UDP example

How node:dgram feels

```
import dgram from 'node:dgram';

const socket = dgram.createSocket('udp4');

socket.on('message', (message, remote) => {
  console.log(`Received ${message.length} bytes from ${remote.address}`);
});

socket.on('error', (error) => {
  console.error(error);
  socket.close();
});

socket.send(Buffer.from('hello'), 9999, '127.0.0.1');
```

send transmits one datagram. message events deliver replies. UDP itself does not promise that a reply will arrive, so our tracker code adds timeouts and retries.

6.3 Parse a tracker URL

The current URL API

```
const trackerUrl = new URL('udp://tracker.example.org:6969/announce');
console.log(trackerUrl.protocol); // udp:
console.log(trackerUrl.hostname); // tracker.example.org
console.log(trackerUrl.port);    // 6969
```

The old tutorial used `url.parse`. `new URL` returns a structured URL object. The port property is text, so `Number(trackerUrl.port)` converts it before `dgram.send`.

6.4 Why UDP trackers use connect then announce

BEP 15 defines two main exchanges for our purpose. The connect request proves that a client can receive packets at its apparent source address and gives it a temporary 64-bit connection ID. The announce request then uses that connection ID and asks for peers for one torrent.

Exchange	Client sends	Tracker returns
Connect	Protocol ID, action 0, random transaction ID.	Action 0, matching transaction ID, 8-byte connection ID.

Exchange	Client sends	Tracker returns
Announce	Connection ID, action 1, transaction ID, info hash, peer ID, transfer counters, event, options, port.	Action 1, matching transaction ID, interval, leechers, seeders, compact peers.

6.5 Transaction IDs

A transaction ID is a random 32-bit value copied into a request. The tracker copies it into the response. This lets the client ignore unrelated packets and match a response to the correct request.

Cryptographically random four-byte value

```
export function randomUInt32() {
  return crypto.randomBytes(4).readUInt32BE(0);
}
```

6.6 Retries

BEP 15 recommends retransmitting after $15 * 2^n$ seconds. The first waits 15 seconds, the next 30, then 60, and so on. This guide tries three times by default. A timeout is expected network behaviour, not necessarily a code exception.

7. Build and parse the UDP connect exchange

7.1 Connect request layout

Offset	Size	Type	Value
0	8 bytes	unsigned 64-bit	Protocol ID 0x41727101980
8	4 bytes	unsigned 32-bit	Action 0 = connect
12	4 bytes	unsigned 32-bit	Random transaction ID

Total size: $8 + 4 + 4 = 16$ bytes.

7.2 Build the request

Connect request builder

```

1 | const UDP_TRACKER_PROTOCOL_ID = 0x41727101980n;
2 | const ACTION_CONNECT = 0;
3 |
4 | export function buildConnectRequest() {
5 |   const transactionId = randomUInt32();
6 |   const buffer = Buffer.alloc(16);
7 |
8 |   buffer.writeBigUInt64BE(UDP_TRACKER_PROTOCOL_ID, 0);
9 |   buffer.writeUInt32BE(ACTION_CONNECT, 8);
10 |   buffer.writeUInt32BE(transactionId, 12);
11 |
12 |   return { buffer, transactionId };
13 | }

```

Line-by-line reasoning

- The n suffix makes the protocol ID a BigInt, so writing all eight bytes is exact.
- Buffer.alloc(16) creates a zero-filled 16-byte packet.
- writeBigUInt64BE writes eight bytes at offset 0.
- writeUInt32BE writes action 0 at offset 8 and the transaction ID at offset 12.
- The function returns both the packet and transaction ID because the parser must later check the reply.

7.3 Connect response layout

Offset	Size	Meaning
0	4	Action. Must be 0 for a successful connect response.
4	4	Transaction ID. Must equal the request ID.
8	8	Connection ID copied into the announce request.

7.4 Parse and validate the response

Connect response parser

```

export function parseConnectResponse(response, expectedTransactionId) {
  ensureTrackerResponse(response, 16, ACTION_CONNECT, expectedTransactionId);

  return {
    action: response.readUInt32BE(0),
    transactionId: response.readUInt32BE(4),
    connectionId: response.subarray(8, 16),
  };
}

```

`subarray(8, 16)` returns the eight connection-ID bytes. We keep them as bytes because the next request only needs to copy them; converting them to a JavaScript Number would be unnecessary and potentially unsafe.

7.5 Generic tracker response validation

Do not trust incoming UDP packets

```
function ensureTrackerResponse(
  response,
  minimumLength,
  expectedAction,
  expectedTransactionId,
) {
  if (response.length < 8) {
    throw new Error('Tracker response is too short.');
```

```
  }

  const action = response.readUInt32BE(0);
  const transactionId = response.readUInt32BE(4);

  if (transactionId !== expectedTransactionId) {
    throw new Error('Tracker response has the wrong transaction ID.');
```

```
  }

  if (action === 3) {
    const message = response.subarray(8).toString('utf8');
    throw new Error(`Tracker error: ${message}`);
  }

  if (action !== expectedAction) {
    throw new Error(`Unexpected tracker action: ${action}`);
  }

  if (response.length < minimumLength) {
    throw new Error('Tracker response is shorter than the protocol requires.');
```

```
  }
}
```

8. Build and parse the UDP announce exchange

8.1 Announce request layout

Offset	Size	Field	Value used by this client
0	8	connection_id	The 8-byte ID returned by connect.
8	4	action	1 = announce.
12	4	transaction_id	A new random 32-bit value.
16	20	info_hash	SHA-1 of the bencoded info dictionary.
36	20	peer_id	Our 20-byte client identifier.
56	8	downloaded	0 for a fresh learning run.
64	8	left	Total bytes still needed; initially the torrent size.
72	8	uploaded	0 because this client does not upload.
80	4	event	2 = started.
84	4	IP address	0 asks the tracker to use the sender address.
88	4	key	A random client key.
92	4	num_want	-1 asks for the tracker default.
96	2	port	6881 by default.

Total size: 98 bytes. Notice that event starts at offset 80 and IP starts at 84. Using offset 80 for the IP, as in the old article code, overwrites the event field.

8.2 Generate one peer ID for the process

src/util.js

```
import { Buffer } from 'node:buffer';
import crypto from 'node:crypto';

const PEER_ID_PREFIX = Buffer.from('-AT0002-', 'ascii');
let peerId;

export function getPeerId() {
  if (!peerId) {
    peerId = Buffer.concat([PEER_ID_PREFIX, crypto.randomBytes(12)]);
  }
  return peerId;
}
```

The prefix is eight bytes and the random suffix is twelve bytes, producing exactly twenty bytes. The same peer ID is reused for tracker and peer messages during one program run. AT is simply an identifier chosen for this tutorial; it is not an officially registered client code.

8.3 Calculate the info hash

20-byte v1 torrent identifier

```
export function infoHash(torrent) {
  const encodedInfo = bencode.encode(torrent.info);
  return crypto.createHash('sha1').update(encodedInfo).digest();
}
```

digest without an encoding argument returns a Buffer. Do not ask for a hex string here, because the protocol requires the raw 20 bytes, not 40 ASCII hex characters.

8.4 Calculate torrent size with BigInt

Single-file and multi-file size calculation

```
export function totalSizeBigInt(torrent) {
  if (torrent.info.files) {
    return torrent.info.files.reduce(
      (total, file) => total + BigInt(file.length),
      0n,
    );
  }
  return BigInt(torrent.info.length);
}
```

The final program rejects multi-file downloads, but calculating their total logical size remains useful and preserves the original tutorial's explanation. 0n is a BigInt zero. Every file length is converted to BigInt before addition.

8.5 Build the announce request

Announce request builder

```
1 | export function buildAnnounceRequest(connectionId, torrent, port = 6881) {
2 |   if (connectionId.length !== 8) {
3 |     throw new Error('A UDP tracker connection ID must contain 8 bytes.');
```

```
4 |   }
5 |
6 |   const transactionId = randomUInt32();
7 |   const buffer = Buffer.alloc(98);
8 |
9 |   connectionId.copy(buffer, 0);
10 |   buffer.writeUInt32BE(1, 8);
11 |   buffer.writeUInt32BE(transactionId, 12);
12 |   infoHash(torrent).copy(buffer, 16);
13 |   getPeerId().copy(buffer, 36);
14 |   buffer.writeBigUInt64BE(0n, 56);
15 |   buffer.writeBigUInt64BE(totalSizeBigInt(torrent), 64);
16 |   buffer.writeBigUInt64BE(0n, 72);
17 |   buffer.writeUInt32BE(2, 80);
18 |   buffer.writeUInt32BE(0, 84);
19 |   buffer.writeUInt32BE(randomUInt32(), 88);
20 |   buffer.writeInt32BE(-1, 92);
21 |   buffer.writeUInt16BE(port, 96);
22 |
23 |   return { buffer, transactionId };
24 | }
```

8.6 Announce response layout

Offset	Size	Field
0	4	Action 1.
4	4	Matching transaction ID.
8	4	Interval in seconds before a normal re-announce.
12	4	Leechers / incomplete peers.
16	4	Seeders / complete peers.
20...	6 per peer	Compact IPv4 peers: 4 address bytes + 2-byte port.

The older article parser read leechers at offset 8 and seeders at 12, skipping interval and shifting the meanings. The corrected parser follows BEP 15.

8.7 Parse compact peers

Correct announce response parser

```
export function parseAnnounceResponse(response, expectedTransactionId) {
  ensureTrackerResponse(response, 20, 1, expectedTransactionId);

  if ((response.length - 20) % 6 !== 0) {
    throw new Error('Tracker returned a malformed compact peer list.');
```

```
  }
```

```
  const peers = [];
```

```
  for (let offset = 20; offset < response.length; offset += 6) {
    peers.push({
      ip: [
        response[offset],
        response[offset + 1],
        response[offset + 2],
        response[offset + 3],
      ].join('.'),
      port: response.readUInt16BE(offset + 4),
    });
  }
```

```
  return {
    action: response.readUInt32BE(0),
    transactionId: response.readUInt32BE(4),
    interval: response.readUInt32BE(8),
    leechers: response.readUInt32BE(12),
    seeders: response.readUInt32BE(16),
    peers,
  };
}
```

8.8 Send, timeout, retry, and match

sendOnce attaches temporary message and error listeners, starts a timer, sends the packet, and resolves only when a response has the correct transaction ID and an expected action. Cleanup removes listeners and the timer so later exchanges do not accidentally process the same packet.

Exponential timeout sequence

```
async function sendWithRetries(options) {
  let lastError;

  for (let attempt = 0; attempt < options.retries; attempt += 1) {
    const timeoutMs = 15_000 * (2 ** attempt);

    try {
      return await sendOnce({ ...options, timeoutMs });
    } catch (error) {
      lastError = error;
      if (error.code !== 'TRACKER_TIMEOUT') throw error;
    }
  }

  throw lastError;
}
```

8.9 The complete getPeers flow

Readable high-level outline

```
export async function getPeers(torrent, options = {}) {
  const trackerUrl = new URL(chooseUdpTracker(torrent));
  const port = Number(trackerUrl.port);
  const socket = dgram.createSocket('udp4');

  try {
    const connectRequest = buildConnectRequest();
    const connectBytes = await sendWithRetries({ /* connect details */ });
    const connect = parseConnectResponse(
      connectBytes,
      connectRequest.transactionId,
    );

    const announceRequest = buildAnnounceRequest(
      connect.connectionId,
      torrent,
      options.listenPort ?? 6881,
    );
```

```
const announceBytes = await sendWithRetries({ /* announce details */ });

return parseAnnounceResponse(
  announceBytes,
  announceRequest.transactionId,
).peers;
} finally {
  socket.close();
}
```

The full version, including all options passed to `sendWithRetries`, appears in Appendix A.

9. TCP, the peer-wire protocol, and the handshake

9.1 A TCP socket in Node.js

Basic peer connection shape

```
import net from 'node:net';

const socket = net.createConnection({ host: peer.ip, port: peer.port });

socket.on('connect', () => {
  console.log('Connected');
});

socket.on('data', (bytes) => {
  console.log(`Received ${bytes.length} bytes`);
});

socket.on('error', console.error);
socket.on('close', () => console.log('Closed'));
```

TCP gives an ordered stream of bytes. It does not preserve the message boundaries used by the BitTorrent application protocol. Chapter 11 builds a framing function to solve that.

9.2 The peer-wire conversation

1. Connect to the peer's IP address and port.
2. Send a BitTorrent handshake containing our torrent info hash and peer ID.
3. Receive and validate the peer's handshake.
4. Send interested.
5. Read choke/unchoke and piece-availability messages.
6. When unchoked, send request messages for blocks the peer has.
7. Receive piece messages and write their block bytes into the output file.

9.3 Handshake layout

Offset	Size	Field	Value
0	1	pstrlen	19
1	19	pstr	ASCII/UTF-8 text BitTorrent protocol
20	8	reserved	All zero in this basic client.
28	20	info_hash	Same raw hash sent to the tracker.
48	20	peer_id	Our process peer ID.

The standard v1 handshake is 68 bytes because $1 + 19 + 8 + 20 + 20 = 68$.

9.4 Build the handshake

Handshake builder

```
1 | const PROTOCOL_NAME = 'BitTorrent protocol';
2 | const PROTOCOL_NAME_BUFFER = Buffer.from(PROTOCOL_NAME, 'utf8');
3 |
4 | export function buildHandshake(torrent) {
5 |   const buffer = Buffer.alloc(68);
6 |
7 |   buffer.writeUInt8(PROTOCOL_NAME_BUFFER.length, 0);
8 |   PROTOCOL_NAME_BUFFER.copy(buffer, 1);
9 |   // Buffer.alloc already made reserved bytes 20-27 zero.
10 |   infoHash(torrent).copy(buffer, 28);
11 |   getPeerId().copy(buffer, 48);
12 |
13 |   return buffer;
```

14 | }

9.5 Parse the peer handshake

Handshake parser

```
export function parseHandshake(buffer) {
  if (buffer.length < 49) {
    throw new Error('Handshake is too short.');
```

The parser does not assume `pstrlen` is 19 while finding offsets; it derives the positions from the first byte. The caller still checks that the protocol text equals BitTorrent protocol and that the returned info hash equals the expected torrent hash.

9.6 Why validate the info hash

A TCP connection can reach the wrong process, a malicious peer, or a peer that closes/reuses a port. Matching the info hash proves that the peer says it is discussing the torrent we requested. `Buffer.equals` compares the raw 20 bytes.

10. Peer message types and message builders

10.1 Common message framing

After the handshake, most peer messages begin with a four-byte big-endian length prefix. The prefix counts every byte after itself: one ID byte plus any payload. A keep-alive is the special case with a zero length and no ID.

ID	Name	Payload
-	keep-alive	No ID and no payload. Four zero bytes.
0	choke	None.
1	unchoke	None.
2	interested	None.
3	not interested	None.
4	have	4-byte piece index.
5	bitfield	One bit for each piece, most-significant bit first within each byte.
6	request	4-byte index + 4-byte begin + 4-byte length.
7	piece	4-byte index + 4-byte begin + block bytes.
8	cancel	Same three integers as request.
9	port	2-byte DHT listen port; not used by this client.

10.2 One generic builder removes repetition

Generic length + ID + payload builder

```
function buildMessage(id, payload = Buffer.alloc(0)) {
  const buffer = Buffer.alloc(4 + 1 + payload.length);
  buffer.writeUInt32BE(1 + payload.length, 0);
  buffer.writeUInt8(id, 4);
  payload.copy(buffer, 5);
  return buffer;
}
```

The old tutorial repeated the allocation and prefix code for every message. A helper is easier to audit: the allocation includes four prefix bytes, one ID byte, and the payload; the stored length excludes the four-byte prefix.

10.3 Messages without payloads

Simple control messages

```
export function buildKeepAlive() {
  return Buffer.alloc(4);
}

export function buildChoke() { return buildMessage(0); }
export function buildUnchoke() { return buildMessage(1); }
export function buildInterested() { return buildMessage(2); }
export function buildUninterested() { return buildMessage(3); }
```

10.4 have and bitfield

Availability messages

```
export function buildHave(pieceIndex) {
  const payload = Buffer.alloc(4);
```

```

    payload.writeUInt32BE(pieceIndex, 0);
    return buildMessage(4, payload);
}

export function buildBitfield(bitfield) {
    return buildMessage(5, bitfield);
}

```

The original article's `buildBitfield` function referred to `payload` even though the parameter was named `bitfield`. The generic helper removes that error and allocates the correct variable length.

10.5 request, piece, and cancel

Block messages

```

export function buildRequest({ index, begin, length }) {
    const payload = Buffer.alloc(12);
    payload.writeUInt32BE(index, 0);
    payload.writeUInt32BE(begin, 4);
    payload.writeUInt32BE(length, 8);
    return buildMessage(6, payload);
}

export function buildPiece({ index, begin, block }) {
    const payload = Buffer.alloc(8 + block.length);
    payload.writeUInt32BE(index, 0);
    payload.writeUInt32BE(begin, 4);
    block.copy(payload, 8);
    return buildMessage(7, payload);
}

export function buildCancel({ index, begin, length }) {
    const payload = Buffer.alloc(12);
    payload.writeUInt32BE(index, 0);
    payload.writeUInt32BE(begin, 4);
    payload.writeUInt32BE(length, 8);
    return buildMessage(8, payload);
}

```

10.6 Parse a complete message

Correctly parse integer length fields

```

export function parseMessage(message) {
    if (message.length < 4) throw new Error('Peer message is too short.');
```

```

    const size = message.readUInt32BE(0);
    if (message.length !== size + 4) {
        throw new Error('Peer message length prefix does not match its data.');
```

```

    }

    if (size === 0) return { size, id: null, payload: null };

    const id = message.readUInt8(4);
    const rawPayload = message.subarray(5);
    let payload = rawPayload;

    if (id === 6 || id === 8) {
        if (rawPayload.length !== 12) throw new Error('Invalid request payload.');
```

```

        payload = {
            index: rawPayload.readUInt32BE(0),
            begin: rawPayload.readUInt32BE(4),
            length: rawPayload.readUInt32BE(8),
        };
    } else if (id === 7) {
        if (rawPayload.length < 8) throw new Error('Invalid piece payload.');
```

```

        payload = {
            index: rawPayload.readUInt32BE(0),
            begin: rawPayload.readUInt32BE(4),
            block: rawPayload.subarray(8),
        };
    }

    return { size, id, payload };
}

```

The older parser stored the remaining four bytes as length for request/cancel instead of reading them as an integer. The modern parser validates exact payload lengths and uses `readUInt32BE`.

11. Reassemble complete messages from a TCP stream

11.1 The common beginner mistake

TCP does not promise this

```
socket.on('data', (data) => {
  // Wrong assumption: data is exactly one complete BitTorrent message.
});
```

A data event can contain half a message, exactly one message, or several messages joined together. This depends on operating-system buffering, packet sizes, timing, and the network. Correct code saves incomplete bytes and repeatedly extracts complete frames.

11.2 Handshake length differs from normal messages

The first incoming application message is the handshake. Its length is `pstrlen + 49`, so we need at least one byte before we know its total length. Later messages store a four-byte length prefix and occupy `prefix + 4` bytes in total.

11.3 Complete framing function

Persistent receive buffer

```
1 | function onWholeMessage(socket, callback) {
2 |   let saved = Buffer.alloc(0);
3 |   let waitingForHandshake = true;
4 |
5 |   socket.on('data', (received) => {
6 |     saved = Buffer.concat([saved, received]);
7 |
8 |     try {
9 |       while (true) {
10 |         if (waitingForHandshake) {
11 |           if (saved.length < 1) return;
12 |
13 |           const handshakeLength = saved.readUInt8(0) + 49;
14 |           if (saved.length < handshakeLength) return;
15 |
16 |           const handshake = saved.subarray(0, handshakeLength);
17 |           saved = saved.subarray(handshakeLength);
18 |           waitingForHandshake = false;
19 |           callback(handshake, true);
20 |           continue;
21 |         }
22 |
23 |         if (saved.length < 4) return;
24 |
25 |         const messageLength = saved.readUInt32BE(0) + 4;
26 |         if (messageLength > 2 * 1024 * 1024) {
27 |           throw new Error('Peer sent an unreasonably large message.');

```

Walkthrough

1. `saved` begins empty and survives between data events because it is closed over by the event callback.
2. New bytes are concatenated after any previously incomplete bytes.
3. During the first frame, one byte reveals the handshake length.
4. If the full frame is not present, return and wait for another data event.
5. When a frame is complete, take it from the front of `saved` and leave the remainder for the next loop iteration.
6. After the handshake, read the normal four-byte length prefix.
7. Reject unreasonable lengths before waiting for or allocating around attacker-controlled values.

8. The while loop means several complete messages in one data event are all delivered to the callback.

12. Pieces, blocks, queues, and failed requests

12.1 Pieces are not normally requested whole

A torrent piece may be hundreds of kilobytes or several megabytes. Peer request messages normally ask for smaller blocks. This guide uses 16 KiB, or $16 * 1024 = 16,384$ bytes, which matches the original tutorial and common protocol practice.

12.2 Piece and block maths

Count expected piece hashes

```
export const BLOCK_LENGTH = 16 * 1024;

export function numberOfPieces(torrent) {
  const count = torrent.info.pieces.length / 20;
  if (!Number.isInteger(count)) {
    throw new Error('The torrent pieces field is not a multiple of 20 bytes.');
```

The final piece may be shorter

```
export function pieceLength(torrent, pieceIndex) {
  const pieceCount = numberOfPieces(torrent);
  const normalLength = BigInt(torrent.info['piece length']);

  if (pieceIndex < pieceCount - 1) return Number(normalLength);

  const bytesBeforeLastPiece = normalLength * BigInt(pieceCount - 1);
  return Number(totalSizeBigInt(torrent) - bytesBeforeLastPiece);
}
```

The final block may also be shorter

```
export function blocksPerPiece(torrent, pieceIndex) {
  return Math.ceil(pieceLength(torrent, pieceIndex) / BLOCK_LENGTH);
}

export function blockLength(torrent, pieceIndex, blockIndex) {
  const begin = blockIndex * BLOCK_LENGTH;
  const remaining = pieceLength(torrent, pieceIndex) - begin;
  return Math.min(BLOCK_LENGTH, remaining);
}
```

12.3 Worked example

Suppose a file is 40,000 bytes and piece length is 32,768. It has two pieces. Piece 0 is 32,768 bytes and contains two 16,384-byte blocks. Piece 1 is 7,232 bytes and contains one short block. The request objects are:

Three block jobs

```
{ index: 0, begin: 0,      length: 16384 }
{ index: 0, begin: 16384, length: 16384 }
{ index: 1, begin: 0,      length: 7232  }
```

12.4 Per-peer Queue

Each peer gets its own queue containing only pieces that peer says it has. Faster peers naturally finish a request and take another job sooner. A Set prevents repeated have messages from adding the same piece twice to one queue.

src/Queue.js

```
export class Queue {
  constructor(torrent) {
    this.torrent = torrent;
```

```

    this.items = [];
    this.queuedPieces = new Set();
    this.choked = true;
  }

  enqueuePiece(pieceIndex) {
    if (this.queuedPieces.has(pieceIndex)) return;
    this.queuedPieces.add(pieceIndex);

    const count = blocksPerPiece(this.torrent, pieceIndex);
    for (let blockIndex = 0; blockIndex < count; blockIndex += 1) {
      this.items.push({
        index: pieceIndex,
        begin: blockIndex * BLOCK_LENGTH,
        length: blockLength(this.torrent, pieceIndex, blockIndex),
      });
    }
  }

  dequeue() { return this.items.shift(); }
  get length() { return this.items.length; }
}

```

12.5 Shared Pieces state

All connections share one Pieces object. It tracks every expected block twice: requested means some peer currently owns the job; received means the block has arrived and been written.

Core of src/Pieces.js

```

export class Pieces {
  constructor(torrent) {
    const makeTable = () =>
      Array.from({ length: numberOfPieces(torrent) }, (_, pieceIndex) =>
        new Array(blocksPerPiece(torrent, pieceIndex)).fill(false),
      );

    this.requested = makeTable();
    this.received = makeTable();
  }

  markRequested(pieceBlock) {
    this.requested[pieceBlock.index][this.blockIndex(pieceBlock)] = true;
  }

  markReceived(pieceBlock) {
    const blockIndex = this.blockIndex(pieceBlock);
    this.requested[pieceBlock.index][blockIndex] = true;
    this.received[pieceBlock.index][blockIndex] = true;
  }

  markFailed(pieceBlock) {
    const blockIndex = this.blockIndex(pieceBlock);
    if (!this.received[pieceBlock.index][blockIndex]) {
      this.requested[pieceBlock.index][blockIndex] = false;
    }
  }

  needed(pieceBlock) {
    return !this.requested[pieceBlock.index][this.blockIndex(pieceBlock)];
  }

  isDone() {
    return this.received.every((blocks) => blocks.every(Boolean));
  }

  blockIndex(pieceBlock) {
    return pieceBlock.begin / BLOCK_LENGTH;
  }
}

```

12.6 Failed requests

If a socket closes while it owns an outstanding block, markFailed changes requested back to false unless the block was already received. Another peer can then take that job. Without this recovery, one dropped connection could leave the download permanently incomplete.

12.7 Choke and unchoke

Each peer starts in the choked state. A choked peer is not currently allowing us to request blocks. The modern code does not close the connection merely because a choke message arrives; it waits. When an unchoke arrives, the queue may resume.

12.8 have and bitfield

A have payload is one piece index. A bitfield packs availability into bits. For each byte, bit 7 represents the first piece in that byte, then bit 6, down to bit 0. That is why the mask is $1 \ll (7 - \text{bitIndex})$.

Translate availability bits into queue jobs

```
function addBitfieldToQueue(bitfield, torrent, queue) {
  const pieceCount = numberOfPieces(torrent);

  bitfield.forEach((byte, byteIndex) => {
    for (let bitIndex = 0; bitIndex < 8; bitIndex += 1) {
      const pieceIndex = byteIndex * 8 + bitIndex;
      if (pieceIndex >= pieceCount) return;

      const mask = 1 << (7 - bitIndex);
      if ((byte & mask) !== 0) queue.enqueuePiece(pieceIndex);
    }
  });
}
```

13. Connect to peers, request blocks, and write the file

13.1 Top-level download state

The download function obtains peers, opens one output file, creates shared piece state, limits the number of simultaneous peer connections, and resolves when all blocks have arrived. A finally block always destroys sockets and closes the file descriptor.

State shared by all connections

```
export async function download(torrent, outputPath) {
  const peers = await getPeers(torrent);
  if (peers.length === 0) throw new Error('The tracker returned no peers.');
```

```
  const pieces = new Pieces(torrent);
  const fileDescriptor = fs.openSync(outputPath, 'w');
  const sockets = new Set();
  let finished = false;

  // Connect to up to 30 peers and wait for completion...
}
```

13.2 Pre-size the output file

Reserve the expected file length when safely representable

```
const totalSize = totalSizeBigInt(torrent);
if (totalSize <= BigInt(Number.MAX_SAFE_INTEGER)) {
  fs.ftruncateSync(fileDescriptor, Number(totalSize));
}
```

Pre-sizing is not required for positional writes, but it makes the intended output size visible immediately. fs APIs use Number offsets in this simple build, so the program avoids converting impossible-to-represent giant values.

13.3 Open a peer connection

One socket, one queue, one outstanding block

```
const socket = net.createConnection({ host: peer.ip, port: peer.port });
const queue = new Queue(torrent);
let inFlight = null;
let connectionError = null;

socket.setTimeout(30_000);

socket.on('connect', () => {
  socket.write(buildHandshake(torrent));
});
```

inFlight remembers the exact request currently assigned to this peer. Limiting each peer to one outstanding request makes the state easy to understand. Real clients pipeline many requests for speed.

13.4 Handle timeout, error, and close

Return abandoned work to the shared pool

```
socket.on('timeout', () => {
  socket.destroy(new Error('Peer connection timed out.));
});

socket.on('error', (error) => {
  connectionError = error;
});

socket.on('close', () => {
  sockets.delete(socket);
  if (inFlight) pieces.markFailed(inFlight);
  peerFinished(connectionError);
});
```

13.5 Dispatch incoming messages

Only the messages needed for downloading

```
switch (parsed.id) {
  case 0:
    queue.choked = true;
    break;
  case 1:
    queue.choked = false;
    requestNextBlock();
    break;
  case 4:
    queue.enqueuePiece(parsed.payload.readUInt32BE(0));
    requestNextBlock();
    break;
  case 5:
    addBitfieldToQueue(parsed.payload, torrent, queue);
    requestNextBlock();
    break;
  case 7:
    handlePiece(parsed.payload);
    break;
  default:
    break;
}
```

Unknown or unused message IDs are ignored in this small client. A production implementation would apply stricter extension negotiation and logging.

13.6 Request the next needed block

Fast peers naturally ask again sooner

```
function requestNextBlock() {
  if (queue.choked || inFlight || isFinished()) return;

  while (queue.length > 0) {
    const pieceBlock = queue.dequeue();

    if (pieces.needed(pieceBlock)) {
      pieces.markRequested(pieceBlock);
      inFlight = pieceBlock;
      socket.write(buildRequest(pieceBlock));
      return;
    }
  }
}
```

A queue item might already have been claimed by another connection since it was enqueued. `pieces.needed` checks the shared state immediately before sending. `markRequested` happens before `socket.write` so two synchronous callers cannot claim the same job in between those operations.

13.7 Validate the piece response

Do not write unexpected data

```
const matchesRequest =
  pieceResponse.index === inFlight.index &&
  pieceResponse.begin === inFlight.begin &&
  pieceResponse.block.length === inFlight.length;

if (!matchesRequest) {
  socket.destroy(new Error('Peer returned a block we did not request.'));
  return;
}
```

A peer controls every incoming byte. The response must match index, begin, and requested length. Closing a misbehaving connection triggers `markFailed`, so the block can be requested elsewhere.

13.8 Calculate the absolute file offset

Piece-relative begin -> whole-file offset

```
const offset =
  pieceResponse.index * torrent.info['piece length'] +
  pieceResponse.begin;
```

`pieceResponse.begin` starts at zero within its piece. Multiplying the piece index by the normal piece length finds the start of the piece in the logical file; adding `begin` finds the block position. This still works for the short final piece because every earlier piece has normal length.

13.9 Write the block

Positional write

```
fs.writeFileSync(
  fileDescriptor,
  pieceResponse.block,
  0,
  pieceResponse.block.length,
  offset,
);
```

Argument	Meaning
<code>fileDescriptor</code>	The open output file returned by <code>fs.openSync</code> .
<code>pieceResponse.block</code>	The received data bytes.
<code>0</code>	Start reading from byte 0 of the block Buffer.
<code>block.length</code>	Write all bytes in this block.
<code>offset</code>	Place them at this absolute output-file position.

Synchronous positional writes are used deliberately to avoid closing the file while asynchronous writes remain pending. The peer sockets are already asynchronous, and the code prioritises clarity over maximum throughput.

13.10 Mark completion and continue

One block completed

```
pieces.markReceived(pieceResponse);
inFlight = null;
process.stdout.write(`Progress: ${pieces.percentDone()}%\r`);

if (pieces.isDone()) {
  finishDownload();
} else {
  requestNextBlock();
}
```

13.11 Finish exactly once

Several sockets may notice completion-related events close together. `finishDownload` checks a shared finished flag, destroys all sockets, and resolves the outer Promise once. The finally block closes the file descriptor after the Promise settles.

14. Run the client safely

14.1 Create index.js

index.js

```

1 | import path from 'node:path';
2 | import process from 'node:process';
3 |
4 | import { download } from './src/download.js';
5 | import {
6 |   isSingleFileTorrent,
7 |   openTorrent,
8 |   torrentName,
9 | } from './src/torrent-parser.js';
10 |
11 | const [torrentPath, requestedOutputPath] = process.argv.slice(2);
12 |
13 | if (!torrentPath) {
14 |   console.error('Usage: node index.js <file.torrent> [output-file]');
15 |   process.exitCode = 1;
16 | } else {
17 |   try {
18 |     const torrent = openTorrent(torrentPath);
19 |
20 |     if (!isSingleFileTorrent(torrent)) {
21 |       throw new Error(
22 |         'This learning client supports single-file torrents only. ' +
23 |         'Multi-file support is explained in the improvements chapter.',
24 |       );
25 |     }
26 |
27 |     const safeDefaultName = path.basename(torrentName(torrent));
28 |     const outputPath = requestedOutputPath ?? safeDefaultName;
29 |
30 |     console.log(`Torrent: ${torrentPath}`);
31 |     console.log(`Saving to: ${outputPath}`);
32 |
33 |     await download(torrent, outputPath);
34 |   } catch (error) {
35 |     console.error(`Download failed: ${error.message}`);
36 |     process.exitCode = 1;
37 |   }
38 | }

```

14.2 Command-line arguments

`process.argv` contains the Node executable, script path, and then user arguments. `slice(2)` removes the first two. The first argument is the `.torrent` path. The optional second argument is the desired output path.

14.3 Safe default file name

`path.basename` removes directory components from the name suggested inside the torrent. This prevents a metainfo name such as `../../something` from choosing a path outside the current directory. A user-supplied output path is still honoured because the user explicitly typed it.

14.4 Choose a suitable legal test torrent

- It must be a BitTorrent v1 single-file torrent.
- It must contain at least one `udp://` tracker in `announce` or `announce-list`.
- Use content you are authorised to download, such as an official open-source operating-system image or your own test data.
- Start with a small file. Public swarms may have no reachable peers, even when your code is correct.

14.5 Run it

Direct Node command

```
node index.js path/to/example.torrent output.bin
```

Equivalent npm script form

```
npm start -- path/to/example.torrent output.bin
```

14.6 Expected output

Illustrative output

```
Torrent: path/to/example.torrent  
Saving to: output.bin  
Tracker returned 18 peer(s).  
Progress: 42%  
...  
Download complete.
```

Peer counts and progress timing vary. A tracker response does not guarantee that every listed peer is online, reachable, unchoking, or still participating.

14.7 Check the result

Compare the final file size with the torrent size. For a robust client, also verify every piece against `info.pieces` while downloading. If an official publisher supplies a whole-file checksum, verify that too, but remember that whole-file checksums are separate from the BitTorrent protocol.

15. Test the protocol code with node:test

15.1 Why test binary layouts

A packet can have the right total length while one field is four bytes too early. Unit tests construct known packets and check exact offsets without depending on a public tracker or peer. This makes debugging much faster.

15.2 Built-in test imports

No third-party test framework required

```
import assert from 'node:assert/strict';
import test from 'node:test';
```

test defines a test case. assert checks expected values and throws a clear failure when they differ.

15.3 Example torrent object

Small deterministic fixture

```
const torrent = {
  announce: Buffer.from('udp://tracker.example:6969'),
  info: {
    name: Buffer.from('example.bin'),
    length: 20_000,
    'piece length': 16_384,
    pieces: Buffer.alloc(40, 1),
  },
};
```

Forty pieces bytes means two 20-byte SHA-1 hashes, so the test torrent has two pieces. The final piece length is 20,000 - 16,384 = 3,616 bytes.

15.4 Test the announce layout

Offsets become executable documentation

```
test('announce request is 98 bytes', () => {
  const connectionId = Buffer.from('0102030405060708', 'hex');
  const request = buildAnnounceRequest(connectionId, torrent, 6881);

  assert.equal(request.buffer.length, 98);
  assert.equal(request.buffer.readBigUInt64BE(64), 20_000n);
  assert.equal(request.buffer.readUInt32BE(80), 2);
  assert.equal(request.buffer.readUInt32BE(84), 0);
  assert.equal(request.buffer.readUInt16BE(96), 6881);
});
```

15.5 Run all tests

Test command

```
npm test
```

The supplied suite checks size and piece maths, handshake construction/parsing, request message parsing, connect response parsing, announce peer parsing, and queue/piece recovery. All six supplied tests passed during preparation of this edition.

16. Common errors and troubleshooting

Symptom	Likely cause and next action
SyntaxError: Cannot use import statement outside a module	package.json is missing "type": "module", or you are running a file outside the project folder. Add the property exactly and run from the folder containing package.json.
ERR_MODULE_NOT_FOUND for a local file	ES module relative imports need the .js extension. Use ./src/download.js, not ./src/download.
Cannot find package bencode	Run npm install in the folder containing package.json. Check that node_modules exists and npm did not report an error.
No UDP tracker was found	The torrent has only HTTP/HTTPS trackers, relies on DHT, or is malformed. Use another legal v1 torrent or implement HTTP tracker support.
UDP tracker did not reply	The tracker may be offline, blocked by a firewall/network, or dropping packets. Try another tracker listed in the torrent. Do not assume the packet builder is wrong until tests pass and multiple trackers fail.
Tracker response has the wrong transaction ID	An unrelated/late UDP packet arrived. Correct code ignores packets while waiting and validates again when parsing.
Tracker returned no peers	The swarm may be empty, the tracker may not know peers, or the info hash may differ because the torrent is non-canonical or v2-only.
All peer connections closed too early	Listed peers may be stale, firewalled, not serving the piece set, or rejecting the basic client. Try another active legal torrent.
Peer connection timed out	The peer stopped responding for 30 seconds. The outstanding block is made available to another peer.
Peer returned a block we did not request	The peer sent a different index, offset, or length. The client closes that untrusted connection.
Downloaded file is corrupt	This educational build does not yet verify completed piece hashes. Implement the integrity step in Chapter 17; also confirm the torrent is v1 single-file and the info hash was calculated from canonical bytes.
EACCES / permission denied	Choose an output folder your user can write to. Do not run the program as administrator merely to bypass a path problem.
Output already exists	Opening with w truncates the previous file. Choose another output path if you need to preserve it.

16.1 Useful diagnostic additions

Print metainfo facts without dumping binary pieces

```
console.log({
  tracker: chooseUdpTracker(torrent),
  name: torrentName(torrent),
  size: totalSizeBigInt(torrent).toString(),
  pieces: numberOfPieces(torrent),
  infoHash: infoHash(torrent).toString('hex'),
});
```

16.2 Check exact Node and package state

Three useful checks

```
node --version
npm ls bencode
npm test
```

17. Improvements and a production-ready roadmap

17.1 Verify every completed piece

This is the most important reliability improvement. When all blocks for a piece arrive, read or assemble that piece, calculate SHA-1, and compare it with the corresponding 20-byte segment of `info.pieces`. Only mark the piece accepted when the hashes match. On mismatch, clear its requested/received state and request every block again from other peers.

Integrity-check core

```
const actual = crypto.createHash('sha1').update(pieceBytes).digest();
const expected = expectedPieceHash(torrent, pieceIndex);

if (!actual.equals(expected)) {
  // Discard/reset this piece and request it again.
}
```

17.2 Preserve the exact raw info dictionary bytes

A production metainfo parser should locate the raw info dictionary byte range and hash it directly. This handles valid-but-non-canonical or unusual files without decode/re-encode ambiguity.

17.3 Multi-file output

Treat torrent data as one logical byte stream, but split each received block across one or more output files based on cumulative file start/end offsets. Sanitise every path component, create folders safely, and reject absolute paths or parent-directory traversal.

17.4 HTTP/HTTPS trackers and compact peers

Implement the standard tracker query with percent-encoded raw `info_hash` and `peer_id` values, parse a bencoded response, and support compact peer lists. Handle tracker failure reason messages and announce intervals.

17.5 DHT, magnet links, and peer exchange

DHT removes the requirement for a central tracker. Magnet links normally identify a torrent by hash and may include tracker hints. Peer exchange lets connected peers share more addresses. Each is a substantial protocol project of its own.

17.6 Better scheduling

- Pipeline several outstanding requests per peer to keep network links busy.
- Use rarest-first selection so uncommon pieces are copied early across the swarm.
- Use end-game mode near completion, where duplicate requests may reduce waiting on one slow peer and are cancelled after the first response.
- Track peer speed, failures, and usefulness; prefer responsive peers.
- Periodically re-announce and add newly discovered peers.

17.7 Resume and persistence

Open an existing file without truncating it, verify saved pieces, and mark valid blocks/pieces complete. Save session state carefully or rebuild it by hashing the file on startup.

17.8 Uploading and fairness

A full BitTorrent client also responds to interested peers, manages choke/unchoke decisions, serves requested blocks, announces uploaded totals, and follows swarm etiquette. Download-only clients contribute nothing and may be deprioritised by peers.

17.9 NAT traversal and inbound connectivity

This guide makes outbound TCP connections only. Full clients may use port forwarding, NAT-PMP, PCP, or UPnP to accept inbound connections. These features change router state and should be implemented with explicit user consent and careful security controls.

17.10 GUI, API, mobile, and WebRTC

- A desktop GUI can call the downloader as a library and display files, peers, speed, and progress.
- A local HTTP API can separate a backend download engine from web, desktop, or mobile interfaces. Add authentication and never expose it publicly by default.
- A mobile client must work within platform background execution, storage, and network restrictions.
- Browser peer-to-peer implementations use WebRTC and the WebTorrent ecosystem rather than raw TCP sockets available to Node.

17.11 BitTorrent v2 and hybrid torrents

BitTorrent v2 changes metainfo structures and uses SHA-256 based Merkle trees. Hybrid torrents carry v1 and v2 data. Do not label this v1 implementation as v2-compatible merely because a file has a .torrent extension.

17.12 Security hardening checklist

- Cap every incoming length before buffering or allocating.
- Validate indexes, offsets, block lengths, peer counts, ports, and file paths.
- Verify piece hashes before accepting data.
- Limit connections, queues, memory use, disk writes, and retries.
- Treat tracker text, peer IDs, and torrent names as untrusted display data.
- Avoid shell execution and never build commands from torrent metadata.
- Use structured logs and tests for malformed packets.

Appendix A. Complete working source code

Create the following files exactly. The code is intentionally compact enough to study, while including validation, tracker retries, TCP framing, failed-request recovery, and tests. The project was syntax-checked and the six supplied unit tests passed during preparation.

package.json

```

1 | {
2 |   "name": "modern-bittorrent-client-tutorial",
3 |   "version": "1.0.0",
4 |   "private": true,
5 |   "description": "A small BitTorrent v1 client built for learning purposes.",
6 |   "type": "module",
7 |   "scripts": {
8 |     "start": "node index.js",
9 |     "test": "node --test"
10 |  },
11 |   "engines": {
12 |     "node": ">=24"
13 |  },
14 |   "dependencies": {
15 |     "bencode": "4.0.1"
16 |  }
17 | }
```

index.js

```

1 | import path from 'node:path';
2 | import process from 'node:process';
3 |
4 | import { download } from './src/download.js';
5 | import {
6 |   isSingleFileTorrent,
7 |   openTorrent,
8 |   torrentName,
9 | } from './src/torrent-parser.js';
10 |
11 | const [torrentPath, requestedOutputPath] = process.argv.slice(2);
12 |
13 | if (!torrentPath) {
14 |   console.error('Usage: node index.js <file.torrent> [output-file]');
15 |   process.exitCode = 1;
16 | } else {
17 |   try {
18 |     const torrent = openTorrent(torrentPath);
19 |
20 |     if (!isSingleFileTorrent(torrent)) {
21 |       throw new Error(
22 |         'This learning client supports single-file torrents only. ' +
23 |         'Multi-file support is explained in the improvements chapter.',
24 |       );
25 |     }
26 |
27 |     const safeDefaultName = path.basename(torrentName(torrent));
28 |     const outputPath = requestedOutputPath ?? safeDefaultName;
29 |
30 |     console.log(`Torrent: ${torrentPath}`);
31 |     console.log(`Saving to: ${outputPath}`);
32 |
33 |     await download(torrent, outputPath);
34 |   } catch (error) {
35 |     console.error(`Download failed: ${error.message}`);
36 |     process.exitCode = 1;
37 |   }
38 | }
```

src/torrent-parser.js

```

1 | import { Buffer } from 'node:buffer';
2 | import crypto from 'node:crypto';
3 | import fs from 'node:fs';
4 |
5 | import bencode from 'bencode';
6 |
7 | export const BLOCK_LENGTH = 16 * 1024;
8 |
```

```

9 | export function openTorrent(filePath) {
10 |   const file = fs.readFileSync(filePath);
11 |   return bencode.decode(file);
12 | }
13 |
14 | export function isSingleFileTorrent(torrent) {
15 |   return torrent.info.files === undefined;
16 | }
17 |
18 | export function torrentName(torrent) {
19 |   return Buffer.from(torrent.info.name).toString('utf8');
20 | }
21 |
22 | export function trackerUrls(torrent) {
23 |   const urls = [];
24 |
25 |   if (torrent.announce) {
26 |     urls.push(Buffer.from(torrent.announce).toString('utf8'));
27 |   }
28 |
29 |   if (Array.isArray(torrent['announce-list'])) {
30 |     for (const tier of torrent['announce-list']) {
31 |       const entries = Array.isArray(tier) ? tier : [tier];
32 |       for (const entry of entries) {
33 |         urls.push(Buffer.from(entry).toString('utf8'));
34 |       }
35 |     }
36 |   }
37 |
38 |   return [...new Set(urls)];
39 | }
40 |
41 | export function chooseUdpTracker(torrent) {
42 |   const url = trackerUrls(torrent).find((candidate) =>
43 |     candidate.startsWith('udp://'),
44 |   );
45 |
46 |   if (!url) {
47 |     throw new Error(
48 |       'No UDP tracker was found in this torrent. ' +
49 |       'This learning client does not yet support HTTP trackers or DHT.',
50 |     );
51 |   }
52 |
53 |   return url;
54 | }
55 |
56 | export function infoHash(torrent) {
57 |   const encodedInfo = bencode.encode(torrent.info);
58 |   return crypto.createHash('sha1').update(encodedInfo).digest();
59 | }
60 |
61 | export function totalSizeBigInt(torrent) {
62 |   if (torrent.info.files) {
63 |     return torrent.info.files.reduce(
64 |       (total, file) => total + BigInt(file.length),
65 |       0n,
66 |     );
67 |   }
68 |
69 |   return BigInt(torrent.info.length);
70 | }
71 |
72 | export function totalSizeBuffer(torrent) {
73 |   const buffer = Buffer.alloc(8);
74 |   buffer.writeBigUInt64BE(totalSizeBigInt(torrent));
75 |   return buffer;
76 | }
77 |
78 | export function numberOfPieces(torrent) {
79 |   const count = torrent.info.pieces.length / 20;
80 |
81 |   if (!Number.isInteger(count)) {
82 |     throw new Error('The torrent pieces field is not a multiple of 20 bytes.');
```

```

103 | }
104 |
105 | export function blocksPerPiece(torrent, pieceIndex) {
106 |   return Math.ceil(pieceLength(torrent, pieceIndex) / BLOCK_LENGTH);
107 | }
108 |
109 | export function blockLength(torrent, pieceIndex, blockIndex) {
110 |   const begin = blockIndex * BLOCK_LENGTH;
111 |   const remaining = pieceLength(torrent, pieceIndex) - begin;
112 |
113 |   if (remaining <= 0) {
114 |     throw new RangeError(
115 |       `Invalid block index ${blockIndex} for piece ${pieceIndex}`,
116 |     );
117 |   }
118 |
119 |   return Math.min(BLOCK_LENGTH, remaining);
120 | }
121 |
122 | export function expectedPieceHash(torrent, pieceIndex) {
123 |   const start = pieceIndex * 20;
124 |   return Buffer.from(torrent.info.pieces).subarray(start, start + 20);
125 | }

```

src/util.js

```

1 | import { Buffer } from 'node:buffer';
2 | import crypto from 'node:crypto';
3 |
4 | const PEER_ID_PREFIX = Buffer.from('-AT0002-', 'ascii');
5 | let peerId;
6 |
7 | export function getPeerId() {
8 |   if (!peerId) {
9 |     peerId = Buffer.concat([PEER_ID_PREFIX, crypto.randomBytes(12)]);
10 |   }
11 |
12 |   return peerId;
13 | }
14 |
15 | export function randomUInt32() {
16 |   return crypto.randomBytes(4).readUInt32BE(0);
17 | }

```

src/message.js

```

1 | import { Buffer } from 'node:buffer';
2 | import { infoHash } from './torrent-parser.js';
3 | import { getPeerId } from './util.js';
4 |
5 | const PROTOCOL_NAME = 'BitTorrent protocol';
6 | const PROTOCOL_NAME_BUFFER = Buffer.from(PROTOCOL_NAME, 'utf8');
7 |
8 | export function buildHandshake(torrent) {
9 |   const buffer = Buffer.alloc(68);
10 |
11 |   buffer.writeUInt8(PROTOCOL_NAME_BUFFER.length, 0);
12 |   PROTOCOL_NAME_BUFFER.copy(buffer, 1);
13 |   // Bytes 20-27 are reserved extension bits. Buffer.alloc() sets them to zero.
14 |   infoHash(torrent).copy(buffer, 28);
15 |   getPeerId().copy(buffer, 48);
16 |
17 |   return buffer;
18 | }
19 |
20 | export function parseHandshake(buffer) {
21 |   if (buffer.length < 49) {
22 |     throw new Error('Handshake is too short.');
```

```

40 | }
41 |
42 | export function buildKeepAlive() {
43 |   return Buffer.alloc(4);
44 | }
45 |
46 | function buildMessage(id, payload = Buffer.alloc(0)) {
47 |   const buffer = Buffer.alloc(4 + 1 + payload.length);
48 |   buffer.writeUInt32BE(1 + payload.length, 0);
49 |   buffer.writeUInt8(id, 4);
50 |   payload.copy(buffer, 5);
51 |   return buffer;
52 | }
53 |
54 | export function buildChoke() {
55 |   return buildMessage(0);
56 | }
57 |
58 | export function buildUnchoke() {
59 |   return buildMessage(1);
60 | }
61 |
62 | export function buildInterested() {
63 |   return buildMessage(2);
64 | }
65 |
66 | export function buildUninterested() {
67 |   return buildMessage(3);
68 | }
69 |
70 | export function buildHave(pieceIndex) {
71 |   const payload = Buffer.alloc(4);
72 |   payload.writeUInt32BE(pieceIndex, 0);
73 |   return buildMessage(4, payload);
74 | }
75 |
76 | export function buildBitfield(bitfield) {
77 |   return buildMessage(5, bitfield);
78 | }
79 |
80 | export function buildRequest({ index, begin, length }) {
81 |   const payload = Buffer.alloc(12);
82 |   payload.writeUInt32BE(index, 0);
83 |   payload.writeUInt32BE(begin, 4);
84 |   payload.writeUInt32BE(length, 8);
85 |   return buildMessage(6, payload);
86 | }
87 |
88 | export function buildPiece({ index, begin, block }) {
89 |   const payload = Buffer.alloc(8 + block.length);
90 |   payload.writeUInt32BE(index, 0);
91 |   payload.writeUInt32BE(begin, 4);
92 |   block.copy(payload, 8);
93 |   return buildMessage(7, payload);
94 | }
95 |
96 | export function buildCancel({ index, begin, length }) {
97 |   const payload = Buffer.alloc(12);
98 |   payload.writeUInt32BE(index, 0);
99 |   payload.writeUInt32BE(begin, 4);
100 |   payload.writeUInt32BE(length, 8);
101 |   return buildMessage(8, payload);
102 | }
103 |
104 | export function buildPort(port) {
105 |   const payload = Buffer.alloc(2);
106 |   payload.writeUInt16BE(port, 0);
107 |   return buildMessage(9, payload);
108 | }
109 |
110 | export function parseMessage(message) {
111 |   if (message.length < 4) {
112 |     throw new Error('Peer message is too short.');
```

```

134 |     payload = {
135 |       index: rawPayload.readUInt32BE(0),
136 |       begin: rawPayload.readUInt32BE(4),
137 |       length: rawPayload.readUInt32BE(8),
138 |     };
139 |   } else if (id === 7) {
140 |     if (rawPayload.length < 8) {
141 |       throw new Error('Piece message has an invalid payload length.');
```

src/tracker.js

```

1 | import { Buffer } from 'node:buffer';
2 | import dgram from 'node:dgram';
3 |
4 | import {
5 |   chooseUdpTracker,
6 |   infoHash,
7 |   totalSizeBigInt,
8 | } from './torrent-parser.js';
9 | import { getPeerId, randomUInt32 } from './util.js';
10 |
11 | const UDP_TRACKER_PROTOCOL_ID = 0x41727101980n;
12 | const ACTION_CONNECT = 0;
13 | const ACTION_ANNOUNCE = 1;
14 | const ACTION_ERROR = 3;
15 |
16 | export async function getPeers(torrent, options = {}) {
17 |   const trackerUrl = new URL(chooseUdpTracker(torrent));
18 |   const port = Number(trackerUrl.port);
19 |   const listenPort = options.listenPort ?? 6881;
20 |   const retries = options.retries ?? 3;
21 |
22 |   if (!port) {
23 |     throw new Error('The UDP tracker URL does not contain a port number.');
```

```

68 | }
69 |
70 | export function buildConnectRequest() {
71 |   const transactionId = randomUInt32();
72 |   const buffer = Buffer.alloc(16);
73 |
74 |   buffer.writeBigUInt64BE(UDP_TRACKER_PROTOCOL_ID, 0);
75 |   buffer.writeUInt32BE(ACTION_CONNECT, 8);
76 |   buffer.writeUInt32BE(transactionId, 12);
77 |
78 |   return { buffer, transactionId };
79 | }
80 |
81 | export function parseConnectResponse(response, expectedTransactionId) {
82 |   ensureTrackerResponse(response, 16, ACTION_CONNECT, expectedTransactionId);
83 |
84 |   return {
85 |     action: response.readUInt32BE(0),
86 |     transactionId: response.readUInt32BE(4),
87 |     connectionId: response.subarray(8, 16),
88 |   };
89 | }
90 |
91 | export function buildAnnounceRequest(connectionId, torrent, port = 6881) {
92 |   if (connectionId.length !== 8) {
93 |     throw new Error('A UDP tracker connection ID must contain 8 bytes.');
```

```

94 |   }
95 |
96 |   const transactionId = randomUInt32();
97 |   const buffer = Buffer.alloc(98);
98 |
99 |   connectionId.copy(buffer, 0);
100 |   buffer.writeUInt32BE(ACTION_ANNOUNCE, 8);
101 |   buffer.writeUInt32BE(transactionId, 12);
102 |   infoHash(torrent).copy(buffer, 16);
103 |   getPeerId().copy(buffer, 36);
104 |   buffer.writeBigUInt64BE(0n, 56); // downloaded
105 |   buffer.writeBigUInt64BE(totalSizeBigInt(torrent), 64); // left
106 |   buffer.writeBigUInt64BE(0n, 72); // uploaded
107 |   buffer.writeUInt32BE(2, 80); // event: started
108 |   buffer.writeUInt32BE(0, 84); // IP address: use the sender's address
109 |   buffer.writeUInt32BE(randomUInt32(), 88); // key
110 |   buffer.writeInt32BE(-1, 92); // number wanted: tracker default
111 |   buffer.writeUInt16BE(port, 96);
112 |
113 |   return { buffer, transactionId };
114 | }
115 |
116 | export function parseAnnounceResponse(response, expectedTransactionId) {
117 |   ensureTrackerResponse(response, 20, ACTION_ANNOUNCE, expectedTransactionId);
118 |
119 |   if ((response.length - 20) % 6 !== 0) {
120 |     throw new Error('Tracker returned a malformed compact peer list.');
```

```

121 |   }
122 |
123 |   const peers = [];
124 |
125 |   for (let offset = 20; offset < response.length; offset += 6) {
126 |     peers.push({
127 |       ip: [
128 |         response[offset],
129 |         response[offset + 1],
130 |         response[offset + 2],
131 |         response[offset + 3],
132 |       ].join('.'),
133 |       port: response.readUInt16BE(offset + 4),
134 |     });
135 |   }
136 |
137 |   return {
138 |     action: response.readUInt32BE(0),
139 |     transactionId: response.readUInt32BE(4),
140 |     interval: response.readUInt32BE(8),
141 |     leechers: response.readUInt32BE(12),
142 |     seeders: response.readUInt32BE(16),
143 |     peers,
144 |   };
145 | }
146 |
147 | function ensureTrackerResponse(
148 |   response,
149 |   minimumLength,
150 |   expectedAction,
151 |   expectedTransactionId,
152 | ) {
153 |   if (response.length < 8) {
154 |     throw new Error('Tracker response is too short.');
```

```

155 |   }
156 |
157 |   const action = response.readUInt32BE(0);
158 |   const transactionId = response.readUInt32BE(4);
159 |
160 |   if (transactionId !== expectedTransactionId) {
161 |     throw new Error('Tracker response has the wrong transaction ID.');
```

```

162 | }
163 |
164 | if (action === ACTION_ERROR) {
165 |   const message = response.subarray(8).toString('utf8');
166 |   throw new Error(`Tracker error: ${message}`);
167 | }
168 |
169 | if (action !== expectedAction) {
170 |   throw new Error(`Unexpected tracker action: ${action}`);
171 | }
172 |
173 | if (response.length < minimumLength) {
174 |   throw new Error('Tracker response is shorter than the protocol requires.');
```

```
175 | }
176 | }
```

```

177 | async function sendWithRetries(options) {
178 |   let lastError;
179 |
180 |   for (let attempt = 0; attempt < options.retries; attempt += 1) {
181 |     const timeoutMs = 15_000 * (2 ** attempt);
182 |
183 |     try {
184 |       return await sendOnce({ ...options, timeoutMs });
185 |     } catch (error) {
186 |       lastError = error;
187 |       if (error.code !== 'TRACKER_TIMEOUT') {
188 |         throw error;
189 |       }
190 |     }
191 |   }
192 | }
193 |
194 | throw lastError;
195 | }
196 |
197 | function sendOnce({
198 |   socket,
199 |   trackerUrl,
200 |   port,
201 |   message,
202 |   transactionId,
203 |   expectedAction,
204 |   timeoutMs,
205 | }) {
206 |   return new Promise((resolve, reject) => {
207 |     let timer;
208 |
209 |     const cleanup = () => {
210 |       clearTimeout(timer);
211 |       socket.off('message', onMessage);
212 |       socket.off('error', onError);
213 |     };
214 |
215 |     const onError = (error) => {
216 |       cleanup();
217 |       reject(error);
218 |     };
219 |
220 |     const onMessage = (response) => {
221 |       if (response.length < 8) return;
222 |       if (response.readUInt32BE(4) !== transactionId) return;
223 |
224 |       const action = response.readUInt32BE(0);
225 |       if (action !== expectedAction && action !== ACTION_ERROR) return;
226 |
227 |       cleanup();
228 |       resolve(response);
229 |     };
230 |
231 |     socket.on('message', onMessage);
232 |     socket.on('error', onError);
233 |
234 |     timer = setTimeout(() => {
235 |       cleanup();
236 |       const error = new Error(
237 |         `UDP tracker did not reply within ${timeoutMs / 1000} seconds.`
238 |       );
239 |       error.code = 'TRACKER_TIMEOUT';
240 |       reject(error);
241 |     }, timeoutMs);
242 |
243 |     socket.send(message, port, trackerUrl.hostname, (error) => {
244 |       if (error) onError(error);
245 |     });
246 |   });
247 | }
```

src/Queue.js

```

1 | import {
2 |   BLOCK_LENGTH,
3 |   blockLength,
4 |   blocksPerPiece,
5 | } from './torrent-parser.js';
6 |
7 | export class Queue {
8 |   constructor(torrent) {
9 |     this.torrent = torrent;
10 |    this.items = [];
11 |    this.queuedPieces = new Set();
12 |    this.choked = true;
13 |  }
14 |
15 |   enqueuePiece(pieceIndex) {
16 |     if (this.queuedPieces.has(pieceIndex)) return;
17 |     this.queuedPieces.add(pieceIndex);
18 |
19 |     const count = blocksPerPiece(this.torrent, pieceIndex);
20 |
21 |     for (let blockIndex = 0; blockIndex < count; blockIndex += 1) {
22 |       this.items.push({
23 |         index: pieceIndex,
24 |         begin: blockIndex * BLOCK_LENGTH,
25 |         length: blockLength(this.torrent, pieceIndex, blockIndex),
26 |       });
27 |     }
28 |   }
29 |
30 |   dequeue() {
31 |     return this.items.shift();
32 |   }
33 |
34 |   get length() {
35 |     return this.items.length;
36 |   }
37 | }

```

src/Pieces.js

```

1 | import {
2 |   BLOCK_LENGTH,
3 |   blocksPerPiece,
4 |   numberOfPieces,
5 | } from './torrent-parser.js';
6 |
7 | export class Pieces {
8 |   constructor(torrent) {
9 |     const makeTable = () =>
10 |       Array.from({ length: numberOfPieces(torrent) }, (_, pieceIndex) =>
11 |         new Array(blocksPerPiece(torrent, pieceIndex)).fill(false),
12 |       );
13 |
14 |     this.requested = makeTable();
15 |     this.received = makeTable();
16 |   }
17 |
18 |   markRequested(pieceBlock) {
19 |     this.requested[pieceBlock.index][this.blockIndex(pieceBlock)] = true;
20 |   }
21 |
22 |   markReceived(pieceBlock) {
23 |     const blockIndex = this.blockIndex(pieceBlock);
24 |     this.requested[pieceBlock.index][blockIndex] = true;
25 |     this.received[pieceBlock.index][blockIndex] = true;
26 |   }
27 |
28 |   markFailed(pieceBlock) {
29 |     const blockIndex = this.blockIndex(pieceBlock);
30 |
31 |     if (!this.received[pieceBlock.index][blockIndex]) {
32 |       this.requested[pieceBlock.index][blockIndex] = false;
33 |     }
34 |   }
35 |
36 |   needed(pieceBlock) {
37 |     const blockIndex = this.blockIndex(pieceBlock);
38 |     return !this.received[pieceBlock.index][blockIndex];
39 |   }
40 |
41 |   isDone() {
42 |     return this.received.every((blocks) => blocks.every(Boolean));
43 |   }
44 |
45 |   percentDone() {
46 |     const receivedBlocks = this.received.flat().filter(Boolean).length;

```

```

47 |     const totalBlocks = this.received.flat().length;
48 |     return Math.floor((receivedBlocks / totalBlocks) * 100);
49 | }
50 |
51 | blockIndex(pieceBlock) {
52 |     return pieceBlock.begin / BLOCK_LENGTH;
53 | }
54 | }

```

src/download.js

```

1 | import { Buffer } from 'node:buffer';
2 | import fs from 'node:fs';
3 | import net from 'node:net';
4 |
5 | import {
6 |     buildHandshake,
7 |     buildInterested,
8 |     buildRequest,
9 |     parseHandshake,
10 |    parseMessage,
11 | } from './message.js';
12 | import { Pieces } from './Pieces.js';
13 | import { Queue } from './Queue.js';
14 | import {
15 |     infoHash,
16 |     numberOfPieces,
17 |     totalSizeBigInt,
18 | } from './torrent-parser.js';
19 | import { getPeers } from './tracker.js';
20 |
21 | const MAX_PEERS = 30;
22 | const MAX_MESSAGE_LENGTH = 2 * 1024 * 1024;
23 |
24 | export async function download(torrent, outputPath) {
25 |     const peers = await getPeers(torrent);
26 |
27 |     if (peers.length === 0) {
28 |         throw new Error('The tracker returned no peers.');
```

```

29 |     }
30 |
31 |     const pieces = new Pieces(torrent);
32 |     const fileDescriptor = fs.openSync(outputPath, 'w');
33 |     const sockets = new Set();
34 |     let finished = false;
35 |
36 |     const totalSize = totalSizeBigInt(torrent);
37 |     if (totalSize <= BigInt(Number.MAX_SAFE_INTEGER)) {
38 |         fs.ftruncateSync(fileDescriptor, Number(totalSize));
39 |     }
40 |
41 |     console.log(`Tracker returned ${peers.length} peer(s).`);
42 |
43 |     try {
44 |         await new Promise((resolve, reject) => {
45 |             let openConnections = 0;
46 |             let lastError;
47 |
48 |             const finishDownload = () => {
49 |                 if (finished) return;
50 |                 finished = true;
51 |                 process.stdout.write('\nDownload complete.\n');
```

```

52 |
53 |                 for (const socket of sockets) {
54 |                     socket.destroy();
55 |                 }
56 |
57 |                 resolve();
58 |             };
59 |
60 |             const peerFinished = (error) => {
61 |                 openConnections -= 1;
62 |                 if (error) lastError = error;
63 |
64 |                 if (!finished && openConnections === 0) {
65 |                     reject(
66 |                         lastError ?? new Error('All peer connections closed too early.'),
67 |                     );
68 |                 }
69 |             };
70 |
71 |             for (const peer of peers.slice(0, MAX_PEERS)) {
72 |                 openConnections += 1;
73 |                 connectToPeer({
74 |                     peer,
75 |                     torrent,
76 |                     pieces,
77 |                     fileDescriptor,
78 |                     sockets,

```

```

79 |         isFinished: () => finished,
80 |         finishDownload,
81 |         peerFinished,
82 |     });
83 |     }
84 | });
85 | } finally {
86 |     for (const socket of sockets) {
87 |         socket.destroy();
88 |     }
89 |     fs.closeSync(fileDescriptor);
90 | }
91 | }
92 |
93 | function connectToPeer(context) {
94 |     const {
95 |         peer,
96 |         torrent,
97 |         pieces,
98 |         fileDescriptor,
99 |         sockets,
100 |         isFinished,
101 |         finishDownload,
102 |         peerFinished,
103 |     } = context;
104 |
105 |     const socket = net.createConnection({ host: peer.ip, port: peer.port });
106 |     const queue = new Queue(torrent);
107 |     let inFlight = null;
108 |     let connectionError = null;
109 |
110 |     sockets.add(socket);
111 |     socket.setTimeout(30_000);
112 |
113 |     socket.on('connect', () => {
114 |         socket.write(buildHandshake(torrent));
115 |     });
116 |
117 |     socket.on('timeout', () => {
118 |         socket.destroy(new Error('Peer connection timed out.'));
119 |     });
120 |
121 |     socket.on('error', (error) => {
122 |         connectionError = error;
123 |     });
124 |
125 |     socket.on('close', () => {
126 |         sockets.delete(socket);
127 |         if (inFlight) pieces.markFailed(inFlight);
128 |         peerFinished(connectionError);
129 |     });
130 |
131 |     onWholeMessage(socket, (message, isHandshakeMessage) => {
132 |         if (isFinished()) return;
133 |
134 |         if (isHandshakeMessage) {
135 |             const handshake = parseHandshake(message);
136 |
137 |             if (handshake.protocol !== 'BitTorrent protocol') {
138 |                 throw new Error('Peer replied with an unknown protocol handshake.');
```

```

173 |         // This small client safely ignores messages it does not use.
174 |         break;
175 |     }
176 | });
177 |
178 | function requestNextBlock() {
179 |     if (queue.choked || inFlight || isFinished()) return;
180 |
181 |     while (queue.length > 0) {
182 |         const pieceBlock = queue.dequeue();
183 |
184 |         if (pieces.needed(pieceBlock)) {
185 |             pieces.markRequested(pieceBlock);
186 |             inFlight = pieceBlock;
187 |             socket.write(buildRequest(pieceBlock));
188 |             return;
189 |         }
190 |     }
191 | }
192 |
193 | function handlePiece(pieceResponse) {
194 |     if (!inFlight) return;
195 |
196 |     const matchesRequest =
197 |         pieceResponse.index === inFlight.index &&
198 |         pieceResponse.begin === inFlight.begin &&
199 |         pieceResponse.block.length === inFlight.length;
200 |
201 |     if (!matchesRequest) {
202 |         socket.destroy(new Error('Peer returned a block we did not request.'));
203 |         return;
204 |     }
205 |
206 |     const offset =
207 |         pieceResponse.index * torrent.info['piece length'] + pieceResponse.begin;
208 |
209 |     fs.writeFileSync(
210 |         fileDescriptor,
211 |         pieceResponse.block,
212 |         0,
213 |         pieceResponse.block.length,
214 |         offset,
215 |     );
216 |
217 |     pieces.markReceived(pieceResponse);
218 |     inFlight = null;
219 |
220 |     process.stdout.write(`Progress: ${pieces.percentDone()}%\r`);
221 |
222 |     if (pieces.isDone()) {
223 |         finishDownload();
224 |     } else {
225 |         requestNextBlock();
226 |     }
227 | }
228 | }
229 |
230 | function addBitfieldToQueue(bitfield, torrent, queue) {
231 |     const pieceCount = numberOfPieces(torrent);
232 |
233 |     bitfield.forEach((byte, byteIndex) => {
234 |         for (let bitIndex = 0; bitIndex < 8; bitIndex += 1) {
235 |             const pieceIndex = byteIndex * 8 + bitIndex;
236 |             if (pieceIndex >= pieceCount) return;
237 |
238 |             const mask = 1 << (7 - bitIndex);
239 |             if ((byte & mask) !== 0) {
240 |                 queue.enqueuePiece(pieceIndex);
241 |             }
242 |         }
243 |     });
244 | }
245 |
246 | function onWholeMessage(socket, callback) {
247 |     let saved = Buffer.alloc(0);
248 |     let waitingForHandshake = true;
249 |
250 |     socket.on('data', (received) => {
251 |         saved = Buffer.concat([saved, received]);
252 |
253 |         try {
254 |             while (true) {
255 |                 if (waitingForHandshake) {
256 |                     if (saved.length < 1) return;
257 |
258 |                     const handshakeLength = saved.readUInt8(0) + 49;
259 |                     if (saved.length < handshakeLength) return;
260 |
261 |                     const handshake = saved.subarray(0, handshakeLength);
262 |                     saved = saved.subarray(handshakeLength);
263 |                     waitingForHandshake = false;
264 |                     callback(handshake, true);
265 |                     continue;
266 |                 }

```

```

267 |         if (saved.length < 4) return;
268 |
269 |         const messageLength = saved.readUInt32BE(0) + 4;
270 |
271 |         if (messageLength > MAX_MESSAGE_LENGTH) {
272 |             throw new Error('Peer sent an unreasonably large message.');
```

test/protocol.test.js

```

1 | import { Buffer } from 'node:buffer';
2 | import assert from 'node:assert/strict';
3 | import test from 'node:test';
4 |
5 | import {
6 |     buildHandshake,
7 |     buildRequest,
8 |     parseHandshake,
9 |     parseMessage,
10 | } from '../src/message.js';
11 | import { Pieces } from '../src/Pieces.js';
12 | import { Queue } from '../src/Queue.js';
13 | import {
14 |     BLOCK_LENGTH,
15 |     blockLength,
16 |     blocksPerPiece,
17 |     infoHash,
18 |     numberOfPieces,
19 |     pieceLength,
20 |     totalSizeBigInt,
21 | } from '../src/torrent-parser.js';
22 | import {
23 |     buildAnnounceRequest,
24 |     buildConnectRequest,
25 |     parseAnnounceResponse,
26 |     parseConnectResponse,
27 | } from '../src/tracker.js';
28 |
29 | const torrent = {
30 |     announce: Buffer.from('udp://tracker.example:6969'),
31 |     info: {
32 |         name: Buffer.from('example.bin'),
33 |         length: 20_000,
34 |         'piece length': 16_384,
35 |         pieces: Buffer.alloc(40, 1),
36 |     },
37 | };
38 |
39 | test('torrent size and piece calculations are correct', () => {
40 |     assert.equal(totalSizeBigInt(torrent), 20_000n);
41 |     assert.equal(numberOfPieces(torrent), 2);
42 |     assert.equal(pieceLength(torrent, 0), 16_384);
43 |     assert.equal(pieceLength(torrent, 1), 3_616);
44 |     assert.equal(blocksPerPiece(torrent, 0), 1);
45 |     assert.equal(blocksPerPiece(torrent, 1), 1);
46 |     assert.equal(blockLength(torrent, 1, 0), 3_616);
47 |     assert.equal(BLOCK_LENGTH, 16_384);
48 | });
49 |
50 | test('handshake is 68 bytes and can be parsed', () => {
51 |     const handshake = buildHandshake(torrent);
52 |     const parsed = parseHandshake(handshake);
53 |     assert.equal(handshake.length, 68);
54 |     assert.equal(parsed.protocol, 'BitTorrent protocol');
55 |     assert.deepEqual(parsed.infoHash, infoHash(torrent));
56 |     assert.equal(parsed.peerId.length, 20);
57 | });
58 |
59 | test('request messages round-trip through the parser', () => {
60 |     const request = { index: 3, begin: 16_384, length: 8_192 };
61 |     const parsed = parseMessage(buildRequest(request));
62 |     assert.equal(parsed.id, 6);
63 |     assert.deepEqual(parsed.payload, request);
64 | });
65 |
66 | test('connect response parser validates and extracts connection id', () => {
```

```

67 | const request = buildConnectRequest();
68 | const response = Buffer.alloc(16);
69 | response.writeUInt32BE(0, 0);
70 | response.writeUInt32BE(request.transactionId, 4);
71 | Buffer.from('0102030405060708', 'hex').copy(response, 8);
72 | const parsed = parseConnectResponse(response, request.transactionId);
73 | assert.equal(parsed.connectionId.toString('hex'), '0102030405060708');
74 | });
75 |
76 | test('announce request is 98 bytes and announce peers parse correctly', () => {
77 |   const connectionId = Buffer.from('0102030405060708', 'hex');
78 |   const request = buildAnnounceRequest(connectionId, torrent, 6881);
79 |   assert.equal(request.buffer.length, 98);
80 |   assert.equal(request.buffer.readBigUInt64BE(64), 20_000n);
81 |
82 |   const response = Buffer.alloc(26);
83 |   response.writeUInt32BE(1, 0);
84 |   response.writeUInt32BE(request.transactionId, 4);
85 |   response.writeUInt32BE(1800, 8);
86 |   response.writeUInt32BE(4, 12);
87 |   response.writeUInt32BE(9, 16);
88 |   Buffer.from([127, 0, 0, 1]).copy(response, 20);
89 |   response.writeUInt16BE(6881, 24);
90 |
91 |   const parsed = parseAnnounceResponse(response, request.transactionId);
92 |   assert.equal(parsed.interval, 1800);
93 |   assert.equal(parsed.leechers, 4);
94 |   assert.equal(parsed.seeders, 9);
95 |   assert.deepEqual(parsed.peers, [{ ip: '127.0.0.1', port: 6881 }]);
96 | });
97 |
98 | test('queue and piece tables describe every block', () => {
99 |   const queue = new Queue(torrent);
100 |   const pieces = new Pieces(torrent);
101 |   queue.enqueuePiece(0);
102 |   queue.enqueuePiece(1);
103 |   assert.equal(queue.length, 2);
104 |   const first = queue.dequeue();
105 |   assert.equal(pieces.needed(first), true);
106 |   pieces.markRequested(first);
107 |   assert.equal(pieces.needed(first), false);
108 |   pieces.markFailed(first);
109 |   assert.equal(pieces.needed(first), true);
110 | });

```

Appendix B. Protocol byte-layout cheat sheets

B.1 UDP tracker connect request - 16 bytes

Bytes	Field	Write method
0-7	Protocol ID	writeBigUInt64BE
8-11	Action 0	writeUInt32BE
12-15	Transaction ID	writeUInt32BE

B.2 UDP tracker connect response - 16 bytes

Bytes	Field	Read method
0-3	Action 0	readUInt32BE
4-7	Transaction ID	readUInt32BE
8-15	Connection ID	subarray(8, 16)

B.3 UDP tracker announce request - 98 bytes

Bytes	Field	Method
0-7	Connection ID	copy
8-11	Action 1	writeUInt32BE
12-15	Transaction ID	writeUInt32BE
16-35	Info hash	copy 20 bytes
36-55	Peer ID	copy 20 bytes
56-63	Downloaded	writeBigUInt64BE
64-71	Left	writeBigUInt64BE
72-79	Uploaded	writeBigUInt64BE
80-83	Event	writeUInt32BE
84-87	IP address	writeUInt32BE
88-91	Key	writeUInt32BE
92-95	Num want	writeInt32BE
96-97	Port	writeUInt16BE

B.4 UDP tracker announce response

Bytes	Field	Method
0-3	Action 1	readUInt32BE
4-7	Transaction ID	readUInt32BE
8-11	Interval	readUInt32BE
12-15	Leechers	readUInt32BE
16-19	Seeders	readUInt32BE
20...	6-byte compact peers	4 raw IP bytes + readUInt16BE port

B.5 Peer handshake - normally 68 bytes

Bytes	Field
0	Protocol string length, normally 19
1-19	BitTorrent protocol
20-27	Reserved extension bits
28-47	Info hash
48-67	Peer ID

B.6 Normal peer message

Bytes	Field
0-3	Length of everything after the prefix
4	Message ID
5...	Payload, if any

B.7 Request message - 17 bytes total

Bytes	Field
0-3	Length = 13
4	ID = 6
5-8	Piece index
9-12	Begin offset inside piece
13-16	Requested block length

B.8 Piece message

Bytes	Field
0-3	Length = 9 + block length
4	ID = 7
5-8	Piece index
9-12	Begin offset inside piece
13...	Block data

Appendix C. Glossary

Term	Meaning
Announce	A tracker request that reports client state and asks for peers.
BEP	BitTorrent Enhancement Proposal, a protocol specification or extension document.
Bencode	The byte-oriented data format used by .torrent metainfo and some BitTorrent messages.
Big-endian	A byte order that stores the most significant byte first.
BigInt	JavaScript integer type for values beyond the exact range of Number.
Bitfield	A compact set of availability flags, one bit per piece.
Block	A requested byte range within a piece.
Buffer	Node.js fixed-length sequence of bytes.
Choke	Peer state that prevents us from making new block requests to that peer.
Compact peer	Six-byte IPv4 peer entry: four address bytes and a two-byte port.
Connection ID	Temporary 8-byte value returned by a UDP tracker connect exchange.
DHT	Distributed hash table used to discover peers without a central tracker.
ES module	Standard JavaScript module format using import and export.
File descriptor	Small operating-system handle used to refer to an open file.
Handshake	First peer-wire message that names the protocol, info hash, and peer ID.
Hash	Fixed-length digest used to identify or verify data.
Have	Peer message declaring possession of one piece index.
Info dictionary	Torrent metadata section whose bencoded bytes define the v1 info hash.
Info hash	20-byte SHA-1 identifier for a BitTorrent v1 torrent.
Keep-alive	Four-byte zero-length peer message used to keep an idle connection open.
Leecher	Peer that does not yet have all data.
Metainfo	The structured content stored in a .torrent file.
NAT	Network address translation, commonly used by home routers.
Offset	Zero-based position in a Buffer, message, piece, or file.
Peer	Another client in the same torrent swarm.
Peer ID	20-byte identifier for a running client.
Piece	Large indexed section of torrent data with an expected hash.
Seeder	Peer that has all torrent data.
SHA-1	20-byte hash function used by BitTorrent v1 info hashes and piece hashes.
Swarm	All peers participating in one torrent.
TCP	Reliable ordered byte-stream transport used for peer connections in this guide.
Tracker	Service that coordinates peer discovery.

Term	Meaning
Transaction ID	Random request value echoed by a UDP tracker response.
UDP	Datagram transport used by the tracker protocol in this guide.
Unchoke	Peer state allowing requests.

Appendix D. Further reading and primary sources

These links are the primary references used to update and verify this edition. Protocol specifications take precedence over tutorial code when they differ.

- [1] Original Allen Kim tutorial: <https://allenkim67.github.io/programming/2016/05/04/how-to-make-your-own-bittorrent-client.html>
- [2] Original Node.js Buffer tutorial: <https://allenkim67.github.io/programming/2016/05/17/nodejs-buffer-tutorial.html>
- [3] Original allen-torrent repository: <https://github.com/allenkim67/allen-torrent>
- [4] Node.js release lines: <https://nodejs.org/en/about/previous-releases>
- [5] Node.js Buffer documentation: <https://nodejs.org/api/buffer.html>
- [6] Node.js dgram documentation: <https://nodejs.org/api/dgram.html>
- [7] Node.js net documentation: <https://nodejs.org/api/net.html>
- [8] Node.js file-system documentation: <https://nodejs.org/api/fs.html>
- [9] Node.js crypto documentation: <https://nodejs.org/api/crypto.html>
- [10] Node.js ECMAScript modules documentation: <https://nodejs.org/api/esm.html>
- [11] Node.js test runner documentation: <https://nodejs.org/api/test.html>
- [12] bencode package repository: <https://github.com/webtorrent/node-bencode>
- [13] BEP 3 - BitTorrent protocol specification: https://www.bittorrent.org/beps/bep_0003.html
- [14] BEP 15 - UDP tracker protocol: https://www.bittorrent.org/beps/bep_0015.html
- [15] BEP 20 - Peer ID conventions: https://www.bittorrent.org/beps/bep_0020.html
- [16] BEP 52 - BitTorrent protocol v2: https://www.bittorrent.org/beps/bep_0052.html

Final learning advice

Build the project in small stages and inspect the actual bytes. When a message fails, compare its layout against the specification field by field. Networking becomes much less mysterious when every offset has a written reason.